

AI Course Recommender System

Machine Learning Recommendation System

Project Overview

This project develops an end-to-end machine learning based course recommendation system for an online learning platform.

The system analyzes course information, course content, learner profiles, and user-course interaction data to recommend relevant courses to learners.

Multiple machine learning approaches are explored, including content-based recommendation, collaborative filtering, supervised learning, unsupervised learning, natural language processing, and deep learning.

The project compares different approaches and evaluates their ability to generate useful and relevant course recommendations.

Contents

1. Business Problem
2. Project Objectives
3. Project Workflow
4. Dataset Loading
5. Dataset Overview
6. Data Quality Analysis
7. Data Cleaning & Preprocessing
8. Exploratory Data Analysis

9. Content-Based Recommendation
10. Collaborative Filtering
11. Recommendation Evaluation
12. Model Comparison and Final Recommendation Strategy
13. Business Insights
14. Limitations
15. Future Improvements
16. Conclusion

1. Business Problem

Online learning platforms can contain thousands of courses, making it difficult for learners to discover courses that match their interests, skills, and learning goals.

A recommendation system can address this problem by analyzing learner behavior and course characteristics to identify courses that are likely to be relevant to each learner.

Problem Statement

Develop a machine learning based recommendation system that can recommend suitable online courses using:

- Course content and genres
- Learner profiles
- User-course interactions
- Course similarity
- Collaborative user behavior
- Machine learning and deep learning techniques

Business Objective

The objective is to improve the learner experience by helping users discover relevant courses efficiently while identifying machine learning approaches that can provide effective recommendations.

2. Project Objectives

The main objectives of this project are:

1. Understand and analyze the course and learner interaction datasets.
2. Perform exploratory data analysis to identify important patterns and trends.
3. Process course text and create machine learning features.
4. Develop content-based recommendation approaches.
5. Develop collaborative filtering recommendation approaches.
6. Apply supervised and unsupervised machine learning techniques where appropriate.
7. Develop a neural-network based recommendation approach.
8. Evaluate and compare the developed models.
9. Analyze recommendation quality and model limitations.
10. Identify useful insights for learners and the online learning platform.
11. Develop a final recommendation strategy based on the experimental results.

3. Project Workflow

The project follows an end-to-end machine learning workflow:

Data Collection & Understanding → Data Cleaning → Exploratory Data Analysis → Feature Engineering → Content-Based Recommendation → Collaborative Filtering → Supervised Machine Learning → Unsupervised Machine Learning → Deep Learning → Model Evaluation → Model Comparison → Recommendation Analysis → Final Recommendation Strategy

In [100...

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

# Display settings
pd.set_option("display.max_columns", None)
pd.set_option("display.max_rows", 100)

# Reproducibility
RANDOM_STATE = 42

print("Libraries imported successfully.")
```

Libraries imported successfully.

```
In [101... course_genre_df = pd.read_csv("course_genre.csv")
course_processed_df = pd.read_csv("course_processed.csv")
ratings_df = pd.read_csv("ratings.csv")

print("Datasets loaded successfully.")
```

Datasets loaded successfully.

```
In [102... df1 = pd.DataFrame(course_genre_df)
df2 = pd.DataFrame(course_processed_df)
df3 = pd.DataFrame(ratings_df)
```

```
In [103... df1
```

```
Out[103...
```

	COURSE_ID	TITLE	Database	Python	CloudComputing	Data/
0	ML0201EN	robots are coming build iot apps with watson ...	0	0	0	
1	ML0122EN	accelerating deep learning with gpu	0	1	0	
2	GPXX0ZG0EN	consuming restful services using the reactive ...	0	0	0	
3	RP0105EN	analyzing big data in r using apache spark	1	0	0	
4	GPXX0Z2PEN	containerizing packaging and running a sprin...	0	0	0	
...	...	...	...	...	...	
302	excourse89	javascript jquery and json	0	0	0	
303	excourse90	programming foundations with javascript html ...	0	0	0	
304	excourse91	front end web development with react	0	0	0	
305	excourse92	introduction to web development	0	0	0	
306	excourse93	interactivity with javascript and jquery	0	0	0	

307 rows × 16 columns



```
In [104... df2
```

Out[104...

	COURSE_ID	TITLE	DESCRIPTION
0	ML0201EN	robots are coming build iot apps with watson ...	have fun with iot and learn along the way if ...
1	ML0122EN	accelerating deep learning with gpu	training complex deep learning models with lar...
2	GPXX0ZG0EN	consuming restful services using the reactive ...	learn how to use a reactive jax rs client to a...
3	RP0105EN	analyzing big data in r using apache spark	apache spark is a popular cluster computing fr...
4	GPXX0Z2PEN	containerizing packaging and running a sprin...	learn how to containerize package and run a ...
...	...	...	...
302	excourse89	javascript jquery and json	in this course we ll look at the javascript l...
303	excourse90	programming foundations with javascript html ...	learn foundational programming concepts e g ...
304	excourse91	front end web development with react	this course explores javascript based front en...
305	excourse92	introduction to web development	this course is designed to start you on a path...
306	excourse93	interactivity with javascript and jquery	this course is the third in our javascript for...

307 rows × 3 columns

In [105...

df3

Out[105...

	user	item	rating
0	1889878	CC0101EN	5
1	1342067	CL0101EN	3
2	1990814	ML0120ENv3	5
3	380098	BD0211EN	5
4	779563	DS0101EN	3
...	...	...	...
233301	1540125	DS0101EN	5
233302	1250651	PY0101EN	5
233303	1003832	CB0105ENv1	3
233304	922065	BD0141EN	4
233305	1596120	DS0301EN	4

233306 rows × 3 columns

In [106...

```
print(df1.shape)
print(df2.shape)
print(df3.shape)
```

(307, 16)

(307, 3)

(233306, 3)

5. Dataset Overview

The project uses three primary datasets:

- `df1` – Course Genre Dataset
- `df2` – Processed Course Dataset
- `df3` – Ratings / User-Course Interaction Dataset

These datasets provide the course information, textual features, and learner interaction data required for the recommendation system.

In [107...

```
print("Course Genre Dataset:", df1.shape)
print("Processed Course Dataset:", df2.shape)
print("Ratings Dataset:", df3.shape)
```

Course Genre Dataset: (307, 16)

Processed Course Dataset: (307, 3)

Ratings Dataset: (233306, 3)

In [108...

```
print("DF1 - Course Genre")
print(df1.columns.tolist())
```

```
print("\nDF2 - Processed Course")
print(df2.columns.tolist())

print("\nDF3 - Ratings")
print(df3.columns.tolist())
```

DF1 - Course Genre

```
['COURSE_ID', 'TITLE', 'Database', 'Python', 'CloudComputing', 'DataAnalysis', 'Containers', 'MachineLearning', 'ComputerVision', 'DataScience', 'BigData', 'Chatbot', 'R', 'BackendDev', 'FrontendDev', 'Blockchain']
```

DF2 - Processed Course

```
['COURSE_ID', 'TITLE', 'DESCRIPTION']
```

DF3 - Ratings

```
['user', 'item', 'rating']
```

In [109...

```
print("DF1 - Course Genre")
display(df1.head())

print("DF2 - Processed Course")
display(df2.head())

print("DF3 - Ratings")
display(df3.head())
```

DF1 - Course Genre

	COURSE_ID	TITLE	Database	Python	CloudComputing	DataAnaly
0	ML0201EN	robots are coming build iot apps with watson ...	0	0	0	
1	ML0122EN	accelerating deep learning with gpu	0	1	0	
2	GPXX0ZG0EN	consuming restful services using the reactive ...	0	0	0	
3	RP0105EN	analyzing big data in r using apache spark	1	0	0	
4	GPXX0Z2PEN	containerizing packaging and running a sprin...	0	0	0	



DF2 - Processed Course

	COURSE_ID	TITLE	DESCRIPTION
0	ML0201EN	robots are coming build iot apps with watson ...	have fun with iot and learn along the way if ...
1	ML0122EN	accelerating deep learning with gpu	training complex deep learning models with lar...
2	GPXX0ZG0EN	consuming restful services using the reactive ...	learn how to use a reactive jax rs client to a...
3	RP0105EN	analyzing big data in r using apache spark	apache spark is a popular cluster computing fr...
4	GPXX0Z2PEN	containerizing packaging and running a sprin...	learn how to containerize package and run a ...

DF3 - Ratings

	user	item	rating
0	1889878	CC0101EN	5
1	1342067	CL0101EN	3
2	1990814	ML0120ENV3	5
3	380098	BD0211EN	5
4	779563	DS0101EN	3

In [110...

```
print("DF1 Data Types")
display(df1.dtypes)

print("DF2 Data Types")
display(df2.dtypes)

print("DF3 Data Types")
display(df3.dtypes)
```

DF1 Data Types

```
COURSE_ID      str
TITLE          str
Database       int64
Python         int64
CloudComputing int64
DataAnalysis   int64
Containers     int64
MachineLearning int64
ComputerVision int64
DataScience    int64
BigData        int64
Chatbot        int64
R              int64
BackendDev     int64
FrontendDev    int64
Blockchain     int64
dtype: object
```

```
DF2 Data Types
COURSE_ID      str
TITLE          str
DESCRIPTION     str
dtype: object
DF3 Data Types
user           int64
item           str
rating         int64
dtype: object
```

```
In [111... print("DF1 Missing Values")
display(df1.isnull().sum())

print("DF2 Missing Values")
display(df2.isnull().sum())

print("DF3 Missing Values")
display(df3.isnull().sum())
```

```
DF1 Missing Values
COURSE_ID      0
TITLE          0
Database       0
Python         0
CloudComputing 0
DataAnalysis   0
Containers     0
MachineLearning 0
ComputerVision 0
DataScience    0
BigData        0
Chatbot        0
R              0
BackendDev     0
FrontendDev    0
Blockchain     0
dtype: int64
DF2 Missing Values
COURSE_ID      0
TITLE          0
DESCRIPTION     0
dtype: int64
DF3 Missing Values
user           0
item           0
rating         0
dtype: int64
```

```
In [112... print("Duplicate Records")

print("DF1:", df1.duplicated().sum())
print("DF2:", df2.duplicated().sum())
print("DF3:", df3.duplicated().sum())
```

Duplicate Records

DF1: 0

DF2: 0

DF3: 0

```
In [113... print("Number of unique courses in DF1:", df1["COURSE_ID"].nunique())

print("Number of unique courses in DF2:", df2["COURSE_ID"].nunique())

print("Number of unique users in DF3:", df3["user"].nunique())

print("Number of unique courses in DF3:", df3["item"].nunique())
```

Number of unique courses in DF1: 307

Number of unique courses in DF2: 307

Number of unique users in DF3: 33901

Number of unique courses in DF3: 126

6. Data Quality Analysis

Data quality is evaluated before preprocessing and modeling.

The analysis includes:

- Missing values
- Duplicate records
- Unique values
- Data types
- Rating distribution
- Course and user coverage
- Potential inconsistencies between datasets

```
In [114... print("DF1 - Course Genre")
df1.info()

print("\nDF2 - Processed Course")
df2.info()

print("\nDF3 - Ratings")
df3.info()
```

```

DF1 - Course Genre
<class 'pandas.DataFrame'>
RangeIndex: 307 entries, 0 to 306
Data columns (total 16 columns):
#   Column                Non-Null Count  Dtype
---  ---
0   COURSE_ID             307 non-null    str
1   TITLE                 307 non-null    str
2   Database               307 non-null    int64
3   Python                307 non-null    int64
4   CloudComputing        307 non-null    int64
5   DataAnalysis          307 non-null    int64
6   Containers            307 non-null    int64
7   MachineLearning       307 non-null    int64
8   ComputerVision        307 non-null    int64
9   DataScience           307 non-null    int64
10  BigData               307 non-null    int64
11  Chatbot               307 non-null    int64
12  R                    307 non-null    int64
13  BackendDev            307 non-null    int64
14  FrontendDev           307 non-null    int64
15  Blockchain            307 non-null    int64
dtypes: int64(14), str(2)
memory usage: 53.8 KB

```

```

DF2 - Processed Course
<class 'pandas.DataFrame'>
RangeIndex: 307 entries, 0 to 306
Data columns (total 3 columns):
#   Column          Non-Null Count  Dtype
---  ---
0   COURSE_ID       307 non-null    str
1   TITLE           307 non-null    str
2   DESCRIPTION      307 non-null    str
dtypes: str(3)
memory usage: 165.5 KB

```

```

DF3 - Ratings
<class 'pandas.DataFrame'>
RangeIndex: 233306 entries, 0 to 233305
Data columns (total 3 columns):
#   Column  Non-Null Count  Dtype
---  ---
0   user    233306 non-null  int64
1   item    233306 non-null  str
2   rating  233306 non-null  int64
dtypes: int64(2), str(1)
memory usage: 7.1 MB

```

```

In [115... missing_df = pd.DataFrame({
    "DF1 Missing": df1.isnull().sum(),
    "DF2 Missing": df2.isnull().sum(),
    "DF3 Missing": df3.isnull().sum()
})

display(missing_df)

```

	DF1 Missing	DF2 Missing	DF3 Missing
BackendDev	0.0	NaN	NaN
BigData	0.0	NaN	NaN
Blockchain	0.0	NaN	NaN
COURSE_ID	0.0	0.0	NaN
Chatbot	0.0	NaN	NaN
CloudComputing	0.0	NaN	NaN
ComputerVision	0.0	NaN	NaN
Containers	0.0	NaN	NaN
DESCRIPTION	NaN	0.0	NaN
DataAnalysis	0.0	NaN	NaN
DataScience	0.0	NaN	NaN
Database	0.0	NaN	NaN
FrontendDev	0.0	NaN	NaN
MachineLearning	0.0	NaN	NaN
Python	0.0	NaN	NaN
R	0.0	NaN	NaN
TITLE	0.0	0.0	NaN
item	NaN	NaN	0.0
rating	NaN	NaN	0.0
user	NaN	NaN	0.0

```
In [116... print("DF1 unique values:")
display(df1.nunique().sort_values(ascending=False))

print("\nDF2 unique values:")
display(df2.nunique().sort_values(ascending=False))

print("\nDF3 unique values:")
display(df3.nunique().sort_values(ascending=False))
```

DF1 unique values:

```

COURSE_ID      307
TITLE          293
Database        2
Python          2
CloudComputing  2
DataAnalysis    2
Containers      2
MachineLearning 2
ComputerVision  2
DataScience     2
BigData         2
Chatbot         2
R               2
BackendDev      2
FrontendDev     2
Blockchain      2
dtype: int64
DF2 unique values:
COURSE_ID      307
DESCRIPTION     299
TITLE          293
dtype: int64
DF3 unique values:
user          33901
item           126
rating         3
dtype: int64

```

In [117...

```

print("Rating values:")
print(sorted(df3["rating"].unique()))

print("\nRating counts:")
display(df3["rating"].value_counts().sort_index())

```

```

Rating values:
[np.int64(3), np.int64(4), np.int64(5)]

```

```

Rating counts:
rating
3      77866
4      77936
5      77504
Name: count, dtype: int64

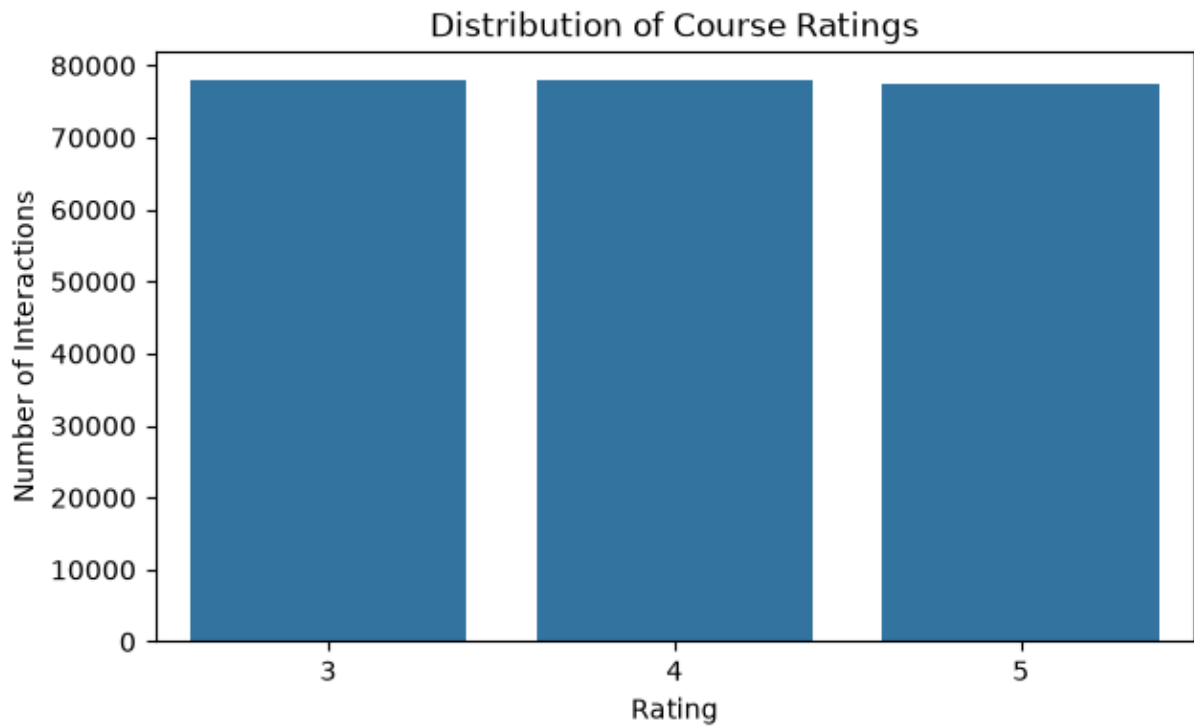
```

In [118...

```

plt.figure(figsize=(7, 4))
sns.countplot(data=df3, x="rating")
plt.title("Distribution of Course Ratings")
plt.xlabel("Rating")
plt.ylabel("Number of Interactions")
plt.savefig("Distribution of Course Ratings")
plt.show()

```



```
In [119...] print("Unique users:", df3["user"].nunique())
print("Unique courses:", df3["item"].nunique())
print("Total interactions:", len(df3))
```

```
Unique users: 33901
Unique courses: 126
Total interactions: 233306
```

```
In [120...] user_interactions = df3.groupby("user").size()

print("Average interactions per user:", round(user_interactions.mean(), 2))
print("Maximum interactions by a user:", user_interactions.max())
print("Minimum interactions by a user:", user_interactions.min())
```

```
Average interactions per user: 6.88
Maximum interactions by a user: 61
Minimum interactions by a user: 1
```

```
In [121...] course_interactions = df3.groupby("item").size()

print("Average interactions per course:", round(course_interactions.mean(), 2))
print("Maximum interactions for a course:", course_interactions.max())
print("Minimum interactions for a course:", course_interactions.min())
```

```
Average interactions per course: 1851.63
Maximum interactions for a course: 14936
Minimum interactions for a course: 1
```

```
In [122...] print("Courses in DF1:", df1["COURSE_ID"].nunique())
print("Courses in DF2:", df2["COURSE_ID"].nunique())
print("Courses in DF3:", df3["item"].nunique())
```

Courses in DF1: 307

Courses in DF2: 307

Courses in DF3: 126

```
In [123... common_courses = set(df1["COURSE_ID"]) & set(df2["COURSE_ID"])

print("Courses common between DF1 and DF2:", len(common_courses))
```

Courses common between DF1 and DF2: 307

```
In [124... interaction_courses = set(df3["item"])

print("Courses in ratings also present in DF1:",
      len(interaction_courses & set(df1["COURSE_ID"])))

print("Courses in ratings not present in DF1:",
      len(interaction_courses - set(df1["COURSE_ID"])))
```

Courses in ratings also present in DF1: 126

Courses in ratings not present in DF1: 0

7. Data Cleaning & Preprocessing

Data preprocessing prepares the datasets for exploratory analysis, feature engineering, and machine learning.

The preprocessing stage includes:

- Handling missing values
- Removing duplicate records where appropriate
- Checking data types
- Standardizing relevant fields
- Validating user and course identifiers
- Preparing the datasets for downstream recommendation models

```
In [125... df1_clean = df1.copy()
df2_clean = df2.copy()
df3_clean = df3.copy()
```

```
In [126... print("DF1 missing values:")
display(df1_clean.isnull().sum())

print("DF2 missing values:")
display(df2_clean.isnull().sum())

print("DF3 missing values:")
display(df3_clean.isnull().sum())
```

DF1 missing values:

```

COURSE_ID      0
TITLE          0
Database        0
Python          0
CloudComputing  0
DataAnalysis    0
Containers      0
MachineLearning 0
ComputerVision  0
DataScience     0
BigData         0
Chatbot         0
R               0
BackendDev      0
FrontendDev     0
Blockchain      0
dtype: int64
DF2 missing values:
COURSE_ID      0
TITLE          0
DESCRIPTION     0
dtype: int64
DF3 missing values:
user           0
item           0
rating         0
dtype: int64

```

```

In [127... print("Duplicates before cleaning:")
print("DF1:", df1_clean.duplicated().sum())
print("DF2:", df2_clean.duplicated().sum())
print("DF3:", df3_clean.duplicated().sum())

```

```

Duplicates before cleaning:
DF1: 0
DF2: 0
DF3: 0

```

```

In [128... display(df1_clean[df1_clean.duplicated(keep=False)].head())
display(df2_clean[df2_clean.duplicated(keep=False)].head())
display(df3_clean[df3_clean.duplicated(keep=False)].head())

```

COURSE_ID

TITLE

Database

Python

CloudComputing

DataAnalysis

Containers

Machine

◀

▶

COURSE_ID

TITLE

DESCRIPTION

user

item

rating

```

In [129... df1_clean = df1_clean.drop_duplicates().reset_index(drop=True)
df2_clean = df2_clean.drop_duplicates().reset_index(drop=True)
df3_clean = df3_clean.drop_duplicates().reset_index(drop=True)

```

```
In [130... print("Duplicates after cleaning:")
print("DF1:", df1_clean.duplicated().sum())
print("DF2:", df2_clean.duplicated().sum())
print("DF3:", df3_clean.duplicated().sum())
```

Duplicates after cleaning:

DF1: 0

DF2: 0

DF3: 0

```
In [131... print("DF1:")
display(df1_clean.dtypes)

print("DF2:")
display(df2_clean.dtypes)

print("DF3:")
display(df3_clean.dtypes)
```

DF1:

COURSE_ID str

TITLE str

Database int64

Python int64

CloudComputing int64

DataAnalysis int64

Containers int64

MachineLearning int64

ComputerVision int64

DataScience int64

BigData int64

Chatbot int64

R int64

BackendDev int64

FrontendDev int64

Blockchain int64

dtype: object

DF2:

COURSE_ID str

TITLE str

DESCRIPTION str

dtype: object

DF3:

user int64

item str

rating int64

dtype: object

```
In [132... print("DF1 Course IDs:", df1_clean["COURSE_ID"].nunique())
print("DF2 Course IDs:", df2_clean["COURSE_ID"].nunique())
print("DF3 Users:", df3_clean["user"].nunique())
print("DF3 Courses:", df3_clean["item"].nunique())
```

DF1 Course IDs: 307
DF2 Course IDs: 307
DF3 Users: 33901
DF3 Courses: 126

```
In [133... print("Minimum rating:", df3_clean["rating"].min())
print("Maximum rating:", df3_clean["rating"].max())
print("Unique ratings:", sorted(df3_clean["rating"].unique()))
```

Minimum rating: 3
Maximum rating: 5
Unique ratings: [np.int64(3), np.int64(4), np.int64(5)]

```
In [134... print("Final dataset sizes after basic preprocessing:")
print("DF1:", df1_clean.shape)
print("DF2:", df2_clean.shape)
print("DF3:", df3_clean.shape)
```

Final dataset sizes after basic preprocessing:
DF1: (307, 16)
DF2: (307, 3)
DF3: (233306, 3)

8. Exploratory Data Analysis

EDA is performed to identify meaningful patterns in course offerings, learner behavior, course popularity, ratings, and user-course interactions.

The analysis will help determine which characteristics of courses and learners can be used to build effective recommendation models.

8. Exploratory Data Analysis Contents

8.1 Course Popularity Analysis

8.2 User Activity Analysis

8.3 Rating Analysis

8.4 User-Course Interaction Analysis

8.5 Course Rating vs. Popularity Analysis

8.6 Course Genre and Learner Interaction Analysis

8.7 Average Rating by Course Genre

8.8 Course Text Analysis

8.9 Course Text Missing-Value Analysis

8.10 Course Title Word Analysis

8.11 Course Title Text Preprocessing

8.12 Stopword Analysis in Course Titles

8.1 Course Popularity Analysis

Course popularity is analyzed using learner interaction data to identify courses that receive the highest number of interactions.

In [135...

```
# Most interacted courses

course_popularity = df3_clean["item"].value_counts()

print("Top 20 most interacted courses:")
display(course_popularity.head(20))
```

Top 20 most interacted courses:

item	
PY0101EN	14936
DS0101EN	14477
BD0101EN	13291
BD0111EN	10599
DA0101EN	8303
DS0103EN	7719
ML0101ENv3	7644
BD0211EN	7551
DS0105EN	7199
BC0101EN	6719
DV0101EN	6709
ML0115EN	6323
CB0103EN	5512
RP0101EN	5237
ST0101EN	5015
CC0101EN	4983
CO0101EN	4480
DB0101EN	3697
BD0115EN	3670
DS0301EN	3624

Name: count, dtype: int64

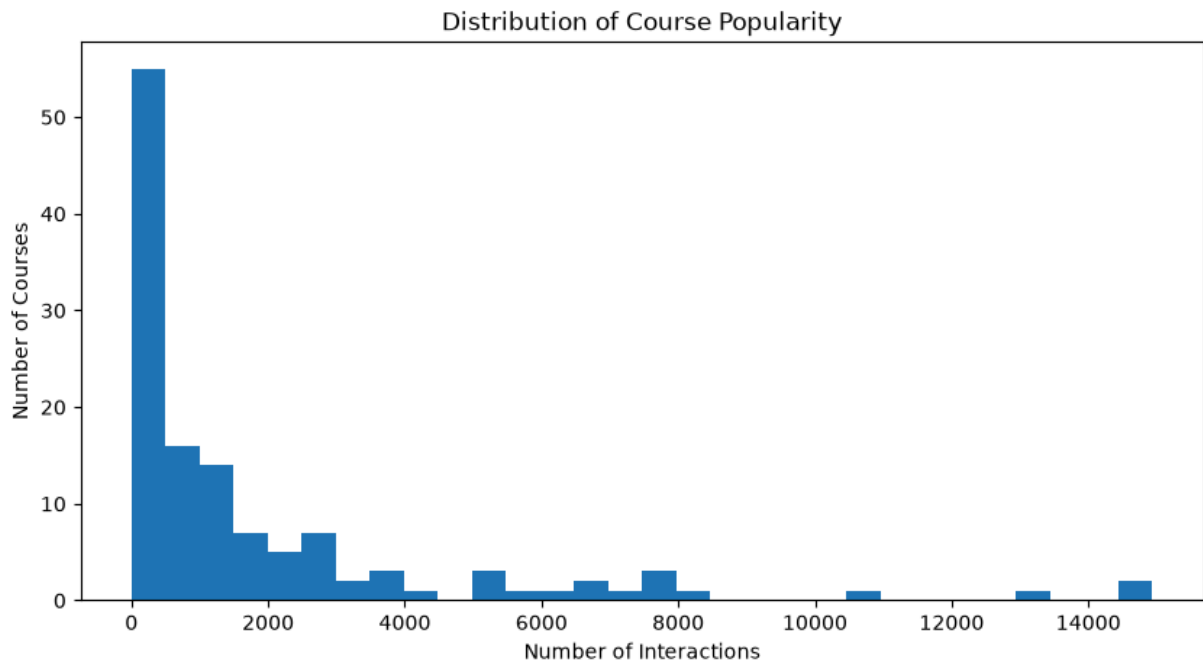
```
In [136... plt.figure(figsize=(10, 5))

plt.hist(course_popularity.values, bins=30)

plt.title("Distribution of Course Popularity")
plt.xlabel("Number of Interactions")
plt.ylabel("Number of Courses")

plt.savefig("course_popularity_distribution.png")

plt.show()
```



```
In [137... top_courses = course_popularity.head(10)

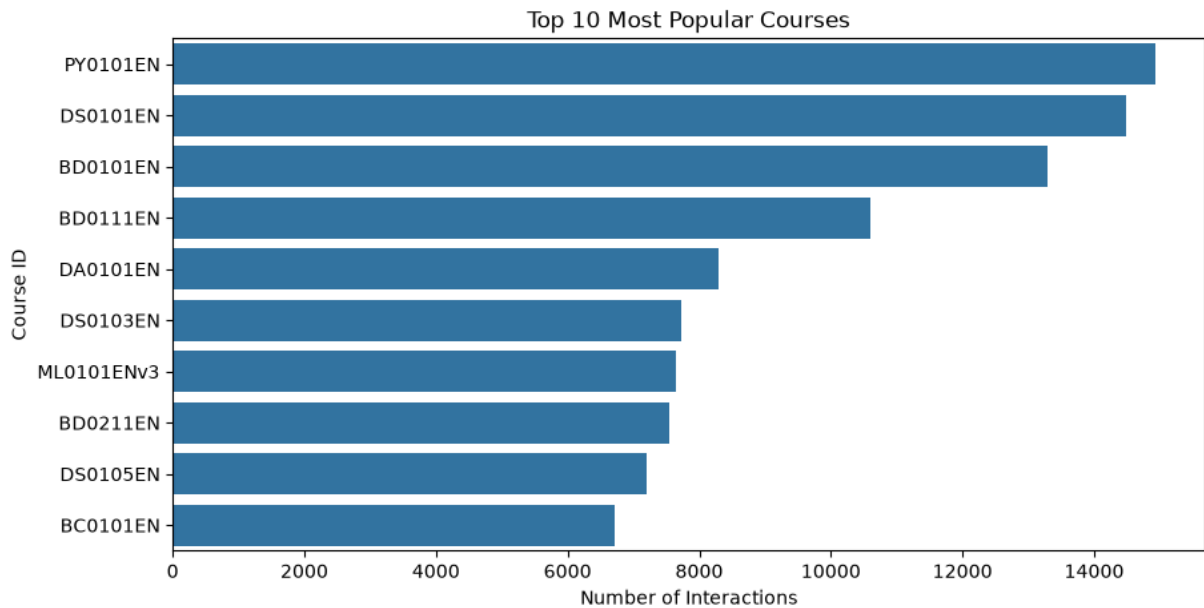
plt.figure(figsize=(10, 5))

sns.barplot(
    x=top_courses.values,
    y=top_courses.index
)

plt.title("Top 10 Most Popular Courses")
plt.xlabel("Number of Interactions")
plt.ylabel("Course ID")

plt.savefig("top_10_popular_courses.png", dpi=300, bbox_inches="tight")

plt.show()
```



In [138...

```
# Least interacted courses

print("Least interacted courses:")
display(course_popularity.sort_values().head(20))
```

Least interacted courses:

item	
DX0108EN	1
DX0106EN	1
ST0301EN	1
GPXX0QR3EN	1
ST0201EN	1
TMP0101EN	2
DX0107EN	2
BENTEST4	2
DS0107	2
TMP107	2
DB0113EN	4
DB0115EN	4
DA0151EN	4
DA0201EN	6
PA0109EN	6
DB0111EN	6
PA0103EN	8
RAVSCTEST1	14
CB0201EN	16
PA0107EN	18

Name: count, dtype: int64

8.2 User Activity Analysis

User activity is analyzed to understand how frequently learners interact with courses.

This analysis helps identify highly active and less active users and provides insight into the level of interaction available for collaborative filtering.

```
In [139... # Number of interactions per user

user_activity = df3_clean["user"].value_counts()

print("Top 20 most active users:")
display(user_activity.head(20))
```

Top 20 most active users:

user

2	61
5	59
924030	51
1653994	51
1559851	50
382781	50
1761291	50
746163	50
971917	50
1207401	49
1114035	49
1823093	49
960869	49
1445103	49
1847957	49
1659210	49
778453	48
507506	48
442078	48
1022021	48

Name: count, dtype: int64

```
In [140... # User activity statistics

print(user_activity.describe())
```

count	33901.000000
mean	6.881980
std	5.823548
min	1.000000
25%	2.000000
50%	6.000000
75%	9.000000
max	61.000000

Name: count, dtype: float64

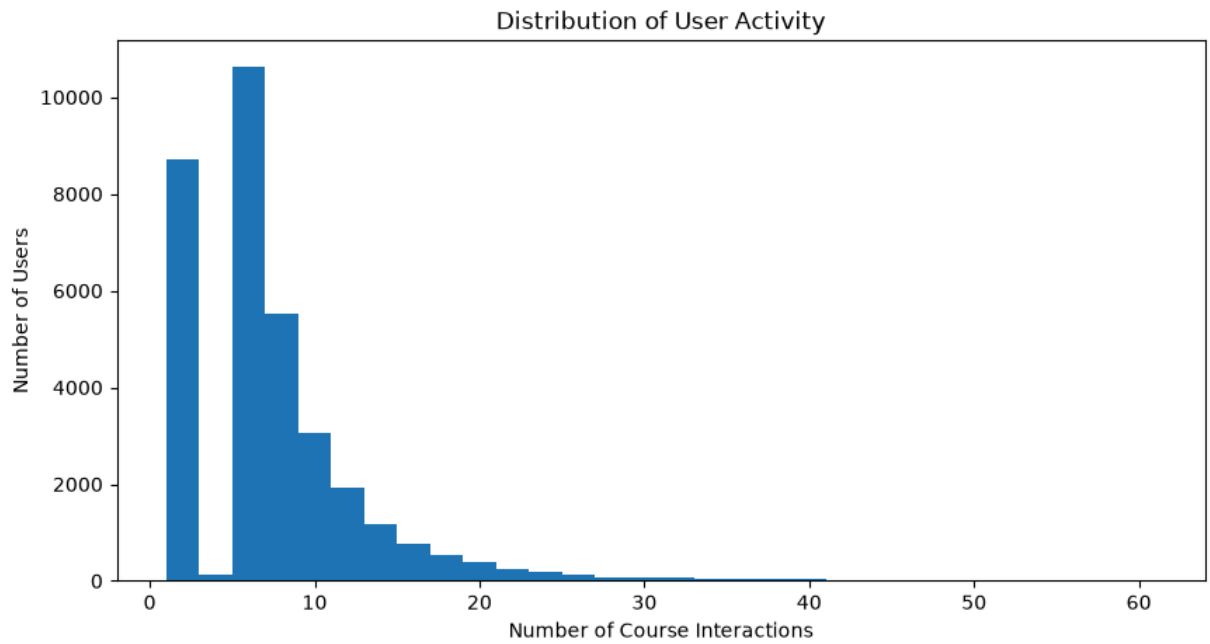
```
In [141... plt.figure(figsize=(10, 5))

plt.hist(user_activity.values, bins=30)

plt.title("Distribution of User Activity")
plt.xlabel("Number of Course Interactions")
plt.ylabel("Number of Users")

plt.savefig("user_activity_distribution.png", dpi=300, bbox_inches="tight")

plt.show()
```



In [142...

```
top_users = user_activity.head(10)

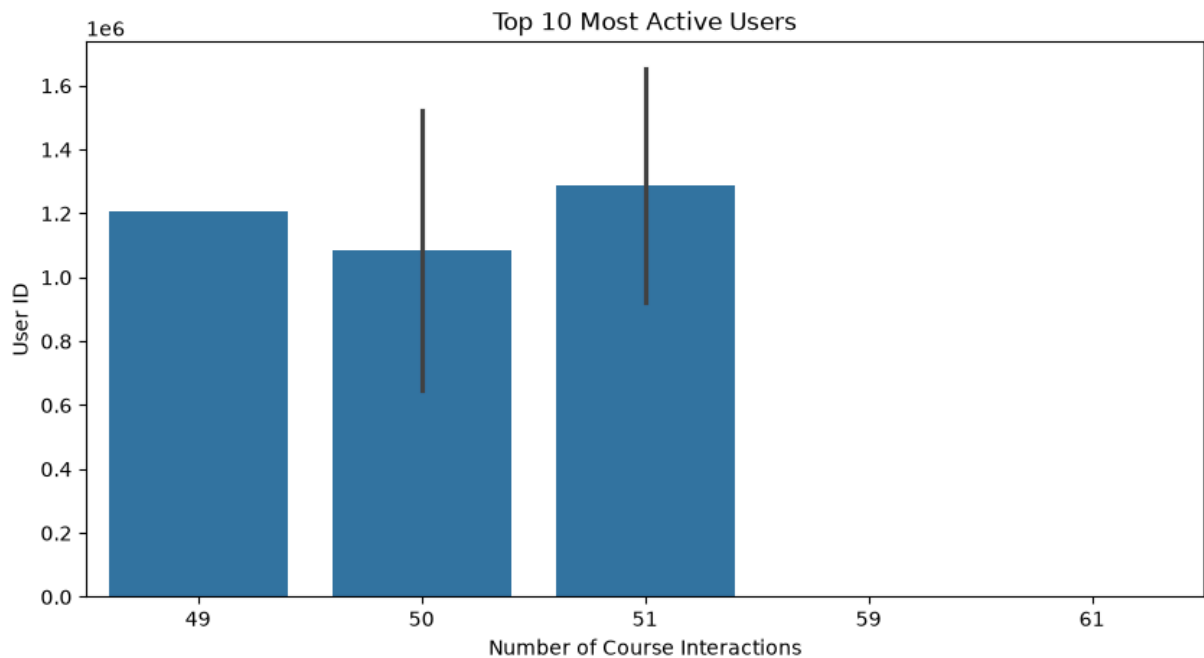
plt.figure(figsize=(10, 5))

sns.barplot(
    x=top_users.values,
    y=top_users.index
)

plt.title("Top 10 Most Active Users")
plt.xlabel("Number of Course Interactions")
plt.ylabel("User ID")

plt.savefig("top_10_active_users.png", dpi=300, bbox_inches="tight")

plt.show()
```



8.3 Rating Analysis

Rating patterns are analyzed to understand learner feedback on courses.

This analysis helps determine the overall rating behavior of learners and whether the ratings are balanced or concentrated around particular values.

In [143...

```
# Rating counts

rating_counts = df3_clean["rating"].value_counts().sort_index()

print(rating_counts)
```

```
rating
3    77866
4    77936
5    77504
Name: count, dtype: int64
```

In [144...

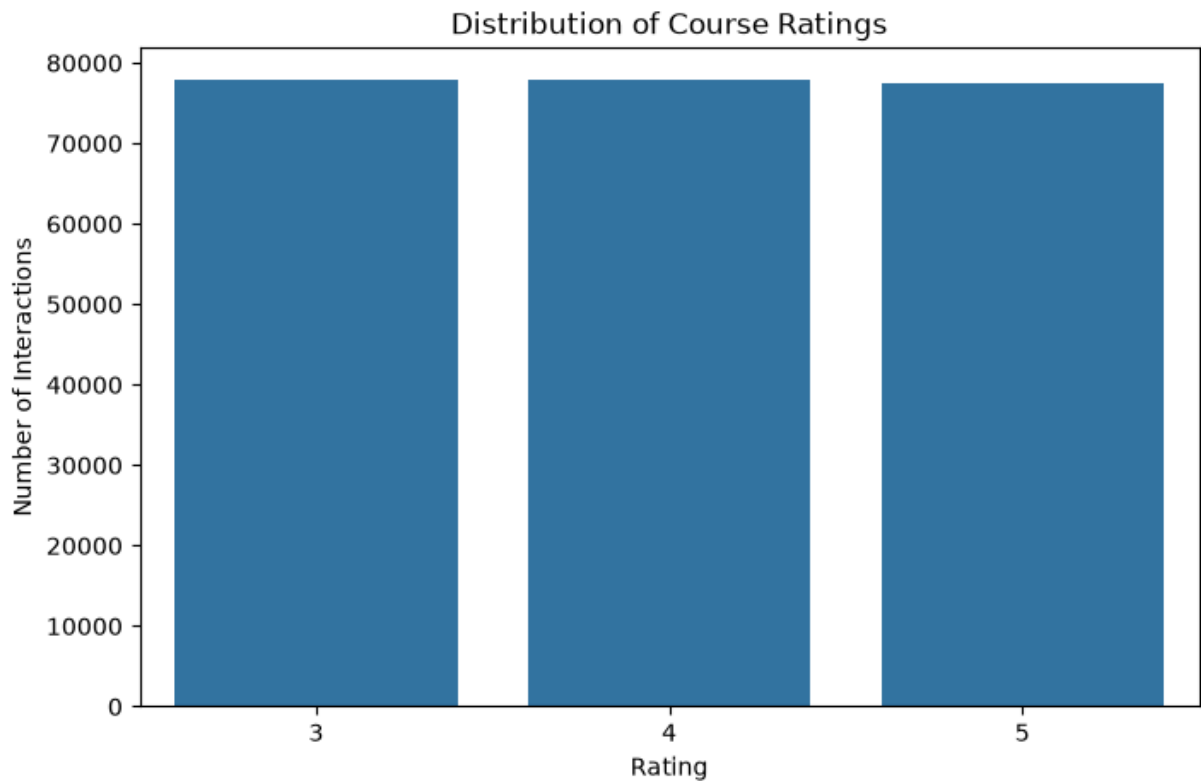
```
plt.figure(figsize=(8, 5))

sns.barplot(
    x=rating_counts.index,
    y=rating_counts.values
)

plt.title("Distribution of Course Ratings")
plt.xlabel("Rating")
plt.ylabel("Number of Interactions")

plt.savefig("rating_distribution.png", dpi=300, bbox_inches="tight")

plt.show()
```



```
In [145... # Rating statistics  
  
print(df3_clean["rating"].describe())
```

```
count    233306.000000  
mean         3.998448  
std         0.816058  
min         3.000000  
25%         3.000000  
50%         4.000000  
75%         5.000000  
max         5.000000  
Name: rating, dtype: float64
```

```
In [146... # Average rating  
  
print("Average rating:", df3_clean["rating"].mean())
```

```
Average rating: 3.998448389668504
```

8.4 User-Course Interaction Analysis

The interaction data is analyzed to understand how learners interact with courses.

This analysis helps identify the density of the user-course interaction matrix and provides important information for collaborative filtering models.

```
In [147... # Number of users and courses  
  
num_users = df3_clean["user"].nunique()
```

```

num_courses = df3_clean["item"].nunique()
num_interactions = len(df3_clean)

print("Number of users:", num_users)
print("Number of courses:", num_courses)
print("Number of interactions:", num_interactions)

```

Number of users: 33901
Number of courses: 126
Number of interactions: 233306

```

In [148... # Interaction density

possible_interactions = num_users * num_courses
interaction_density = (num_interactions / possible_interactions) * 100

print("Possible interactions:", possible_interactions)
print("Actual interactions:", num_interactions)
print("Interaction density: {:.4f}%".format(interaction_density))

```

Possible interactions: 4271526
Actual interactions: 233306
Interaction density: 5.4619%

```

In [149... # Average interactions per user

avg_user_interactions = df3_clean.groupby("user")["item"].count().mean()

print("Average interactions per user:", avg_user_interactions)

```

Average interactions per user: 6.8819798825993335

```

In [150... # Average interactions per course

avg_course_interactions = df3_clean.groupby("item")["user"].count().mean()

print("Average interactions per course:", avg_course_interactions)

```

Average interactions per course: 1851.6349206349207

```

In [151... # Sample interaction matrix

interaction_matrix = df3_clean.pivot_table(
    index="user",
    columns="item",
    values="rating",
    aggfunc="mean"
)

plt.figure(figsize=(12, 8))

sns.heatmap(
    interaction_matrix.iloc[:30, :30],
)

plt.title("User-Course Interaction Heatmap")
plt.xlabel("Course ID")
plt.ylabel("User ID")

```

```
plt.savefig("user_course_interaction_heatmap.png",
            dpi=300,
            bbox_inches="tight")

plt.show()
```



8.5 Course Rating vs. Popularity Analysis

Course ratings and interaction counts are compared to understand whether highly interacted courses also tend to receive higher ratings.

This analysis helps distinguish course popularity from learner satisfaction.

In [152... *# Average rating and interaction count for each course*

```
course_stats = df3_clean.groupby("item").agg(
    average_rating=("rating", "mean"),
    interaction_count=("rating", "count")
).reset_index()

display(course_stats.head(10))
```

	item	average_rating	interaction_count
0	AI0111EN	3.890339	383
1	BC0101EN	3.998661	6719
2	BC0201EN	4.004187	2866
3	BC0202EN	3.997419	775
4	BD0101EN	3.997141	13291
5	BD0111EN	4.003114	10599
6	BD0115EN	3.998638	3670
7	BD0121EN	3.986940	1608
8	BD0123EN	3.978471	1347
9	BD0131EN	3.975635	2873

In [153...

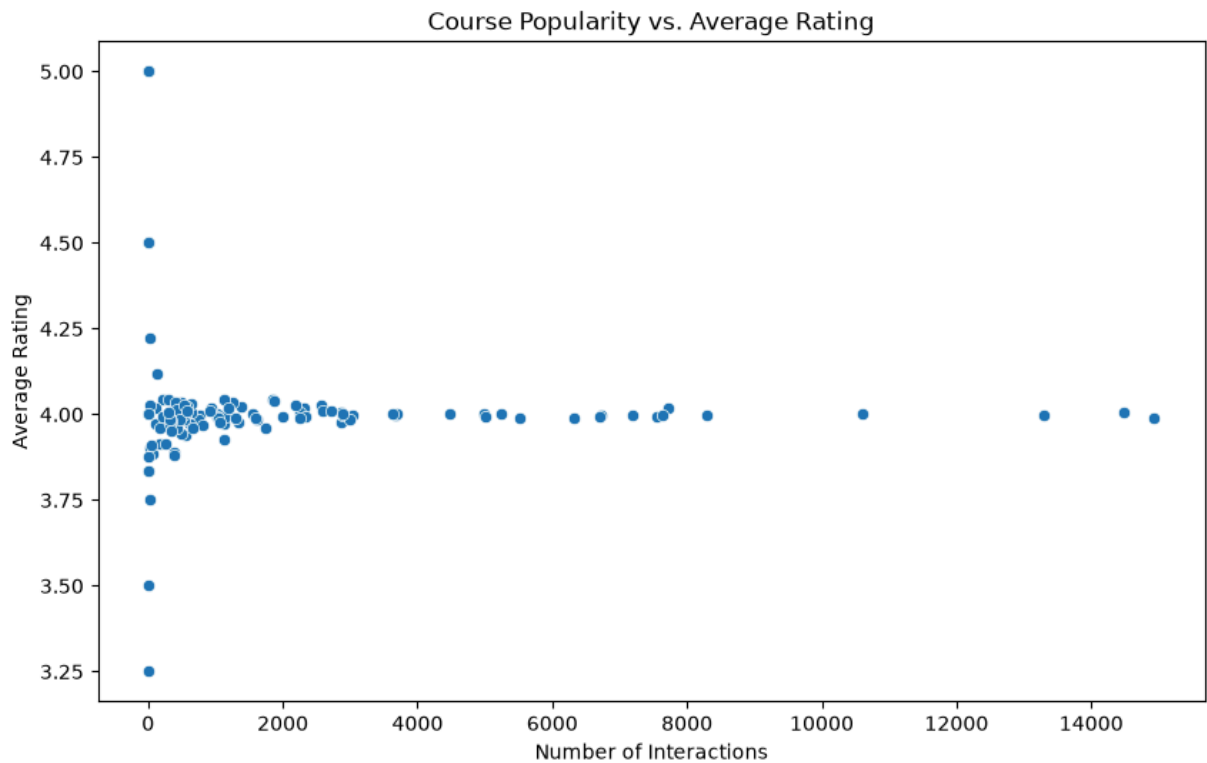
```
plt.figure(figsize=(10, 6))

sns.scatterplot(
    data=course_stats,
    x="interaction_count",
    y="average_rating"
)

plt.title("Course Popularity vs. Average Rating")
plt.xlabel("Number of Interactions")
plt.ylabel("Average Rating")

plt.savefig("course_popularity_vs_rating.png",
            dpi=300,
            bbox_inches="tight")

plt.show()
```



```
In [154... # Correlation between popularity and average rating

correlation = course_stats["interaction_count"].corr(
    course_stats["average_rating"]
)

print("Correlation between course popularity and average rating:",
      correlation)
```

Correlation between course popularity and average rating: -0.04058395147880707

```
In [155... # Courses with the highest average ratings

topRatedCourses = course_stats.sort_values(
    "average_rating",
    ascending=False
).head(10)

display(topRatedCourses)
```

	item	average_rating	interaction_count
66	DX0106EN	5.000000	1
68	DX0108EN	5.000000	1
114	ST0301EN	5.000000	1
22	BENTEST4	4.500000	2
48	DB0113EN	4.500000	4
98	PA0107EN	4.222222	18
16	BD0151EN	4.120567	141
106	RP0105EN	4.045374	1124
35	CNSC02EN	4.043526	1838
40	COM001EN	4.043062	209

8.6 Course Genre and Learner Interaction Analysis

The course genre information is combined with learner interaction data to understand which course categories receive the most learner activity.

This analysis helps identify differences between the number of courses available in a genre and the level of learner interest in that genre.

In [156...

```
# Define course genre columns

genre_columns = [
    "Database",
    "Python",
    "CloudComputing",
    "DataAnalysis",
    "Containers",
    "MachineLearning",
    "ComputerVision",
    "DataScience",
    "BigData",
    "Chatbot",
    "R",
    "BackendDev",
    "FrontendDev",
    "Blockchain"
]

print("Number of genres:", len(genre_columns))
```

Number of genres: 14

In [157...

```
# Number of courses in each genre
```

```
genre_counts = df1_clean[genre_columns].sum().sort_values(ascending=False)

print(genre_counts)
```

```
BackendDev      78
MachineLearning 69
Database        60
DataAnalysis    56
DataScience     38
CloudComputing  37
BigData         36
Python          28
FrontendDev     19
Containers      17
R              17
ComputerVision  10
Chatbot         4
Blockchain      4
dtype: int64
```

```
In [158... plt.figure(figsize=(12, 6))

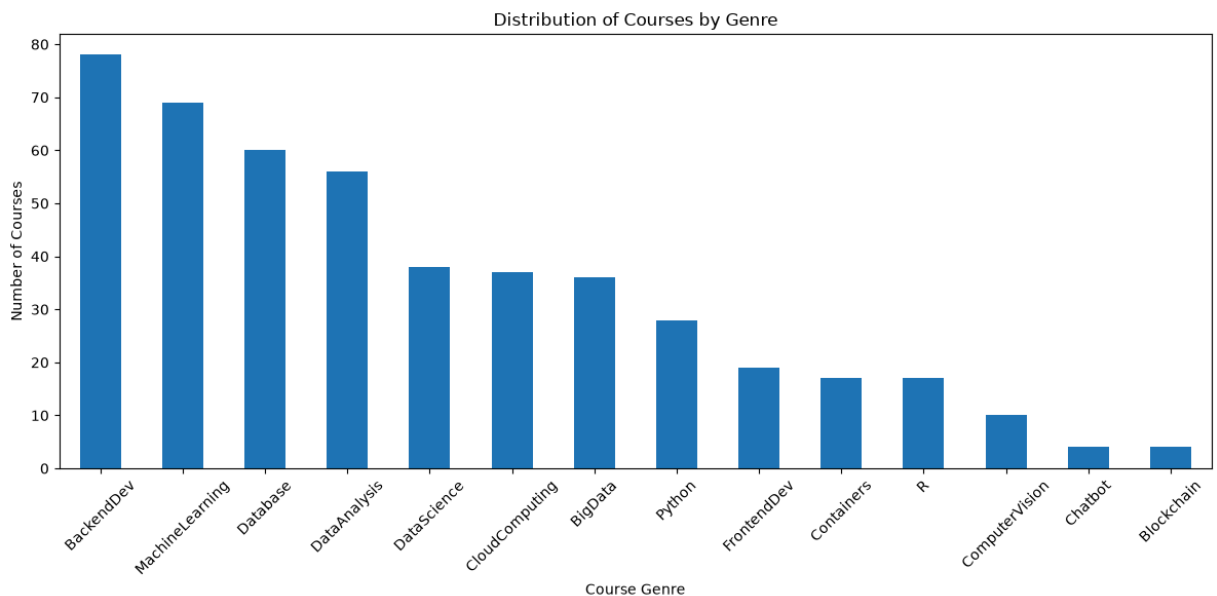
genre_counts.plot(kind="bar")

plt.title("Distribution of Courses by Genre")
plt.xlabel("Course Genre")
plt.ylabel("Number of Courses")
plt.xticks(rotation=45)

plt.tight_layout()

plt.savefig("course_genre_distribution.png", dpi=300, bbox_inches="tight")

plt.show()
```



```
In [159... # Merge course genre information with interaction data

genre_interactions = df3_clean.merge(
```

```

df1_clean,
left_on="item",
right_on="COURSE_ID",
how="left"
)

print("Merged dataset shape:", genre_interactions.shape)

display(genre_interactions.head())

```

Merged dataset shape: (233306, 19)

	user	item	rating	COURSE_ID	TITLE	Database	Python	CloudCompu
0	1889878	CC0101EN	5	CC0101EN	introduction to cloud	0	0	
1	1342067	CL0101EN	3	CL0101EN	ibm cloud essentials	0	0	
2	1990814	ML0120ENV3	5	ML0120ENV3	deep learning with tensorflow	0	0	
3	380098	BD0211EN	5	BD0211EN	spark fundamentals i	1	0	
4	779563	DS0101EN	3	DS0101EN	introduction to data science	0	0	

In [160...

```

# Number of Learner interactions for each genre

genre_interactions_count = (
    genre_interactions[genre_columns]
    .multiply(genre_interactions["rating"], axis=0)
    .sum()
    .sort_values(ascending=False)
)

print(genre_interactions_count)

```

Database	251151
DataScience	233712
BigData	216282
DataAnalysis	164981
Python	158158
MachineLearning	140797
CloudComputing	108985
BackendDev	102633
R	46578
Blockchain	45993
Containers	45258
Chatbot	22932
FrontendDev	14254
ComputerVision	78

dtype: int64

In [161...

```
plt.figure(figsize=(12, 6))

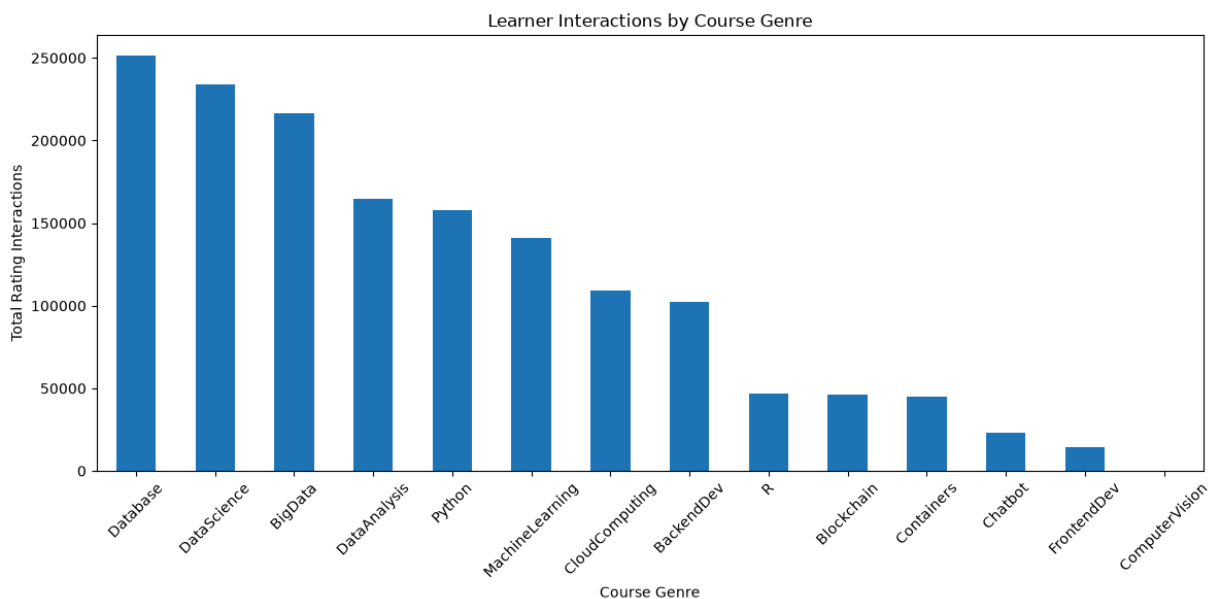
genre_interactions_count.plot(kind="bar")

plt.title("Learner Interactions by Course Genre")
plt.xlabel("Course Genre")
plt.ylabel("Total Rating Interactions")
plt.xticks(rotation=45)

plt.tight_layout()

plt.savefig("genre_interaction_distribution.png", dpi=300, bbox_inches="tight")

plt.show()
```



8.7 Average Rating by Course Genre

The average learner rating is calculated separately for each course genre.

This helps identify which learning areas receive more positive learner feedback, independent of the total number of interactions.

```
In [162... # Average rating for each genre

genre_average_ratings = {}

for genre in genre_columns:
    genre_data = genre_interactions[
        genre_interactions[genre] == 1
    ]

    genre_average_ratings[genre] = genre_data["rating"].mean()

genre_average_ratings = pd.Series(
    genre_average_ratings
).sort_values(ascending=False)

print(genre_average_ratings)
```

```
R          4.015691
Containers  4.008325
DataScience 4.001164
CloudComputing 4.000771
Blockchain  3.999391
MachineLearning 3.997643
Database    3.997119
DataAnalysis 3.996246
BigData     3.996194
BackendDev  3.995056
Python      3.992377
Chatbot     3.987480
FrontendDev 3.977121
ComputerVision 3.900000
dtype: float64
```

```
In [163... plt.figure(figsize=(12, 6))

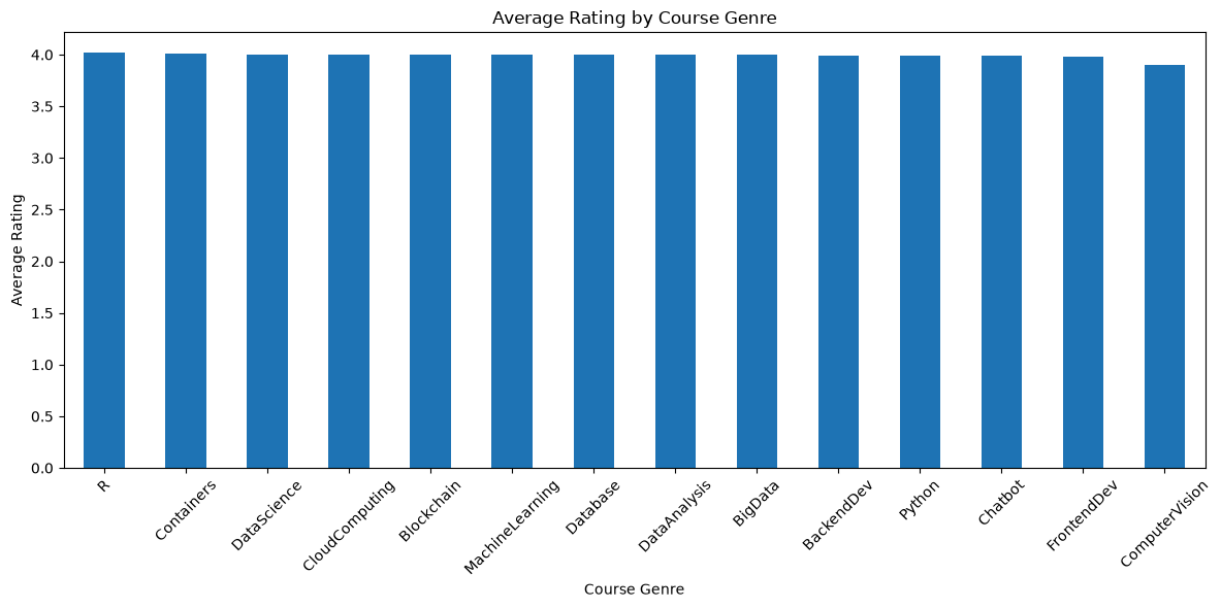
genre_average_ratings.plot(kind="bar")

plt.title("Average Rating by Course Genre")
plt.xlabel("Course Genre")
plt.ylabel("Average Rating")
plt.xticks(rotation=45)

plt.tight_layout()

plt.savefig("average_rating_by_genre.png", dpi=300, bbox_inches="tight")

plt.show()
```



```
In [164... print("Top 5 highest-rated genres:")

display(
    genre_average_ratings.head(5)
)
```

```
Top 5 highest-rated genres:
R                4.015691
Containers       4.008325
DataScience      4.001164
CloudComputing   4.000771
Blockchain       3.999391
dtype: float64
```

```
In [165... print("Bottom 5 lowest-rated genres:")

display(
    genre_average_ratings.tail(5)
)
```

```
Bottom 5 lowest-rated genres:
BackendDev       3.995056
Python           3.992377
Chatbot          3.987480
FrontendDev      3.977121
ComputerVision   3.900000
dtype: float64
```

8.8 Course Text Analysis

The processed course dataset contains course titles and descriptions.

The length of these text fields is analyzed to understand the amount of textual information available for NLP-based recommendation methods such as Bag of Words and TF-IDF.

In [166...

```
# Calculate text length

df2_clean["title_length"] = df2_clean["TITLE"].fillna("").str.len()
df2_clean["description_length"] = df2_clean["DESCRIPTION"].fillna("").str.len()

display(
    df2_clean[[
        "COURSE_ID",
        "TITLE",
        "title_length",
        "description_length"
    ]].head()
)
```

	COURSE_ID	TITLE	title_length	description_length
0	ML0201EN	robots are coming build iot apps with watson ...	66	367
1	ML0122EN	accelerating deep learning with gpu	35	1286
2	GPXX0ZG0EN	consuming restful services using the reactive ...	59	99
3	RP0105EN	analyzing big data in r using apache spark	42	221
4	GPXX0Z2PEN	containerizing packaging and running a sprin...	64	117

In [167...

```
# Text Length statistics

print("Title length statistics:")
print(df2_clean["title_length"].describe())

print("\nDescription length statistics:")
print(df2_clean["description_length"].describe())
```

```
Title length statistics:
count    307.000000
mean     41.859935
std      20.228028
min       5.000000
25%      27.000000
50%      39.000000
75%      53.000000
max     120.000000
Name: title_length, dtype: float64
```

```
Description length statistics:
count    307.000000
mean    476.276873
std     529.703862
min     15.000000
25%    118.500000
50%    248.000000
75%    698.000000
max   3093.000000
Name: description_length, dtype: float64
```

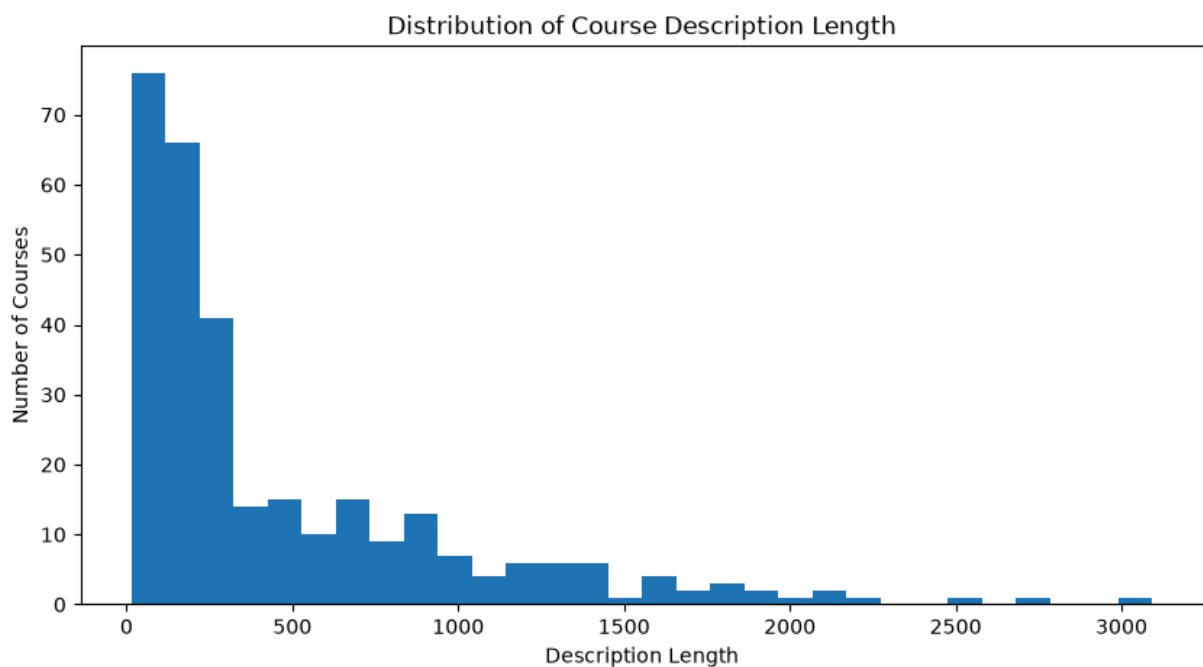
```
In [168... plt.figure(figsize=(10, 5))

plt.hist(df2_clean["description_length"], bins=30)

plt.title("Distribution of Course Description Length")
plt.xlabel("Description Length")
plt.ylabel("Number of Courses")

plt.savefig("course_description_length_distribution.png",
            dpi=300,
            bbox_inches="tight")

plt.show()
```



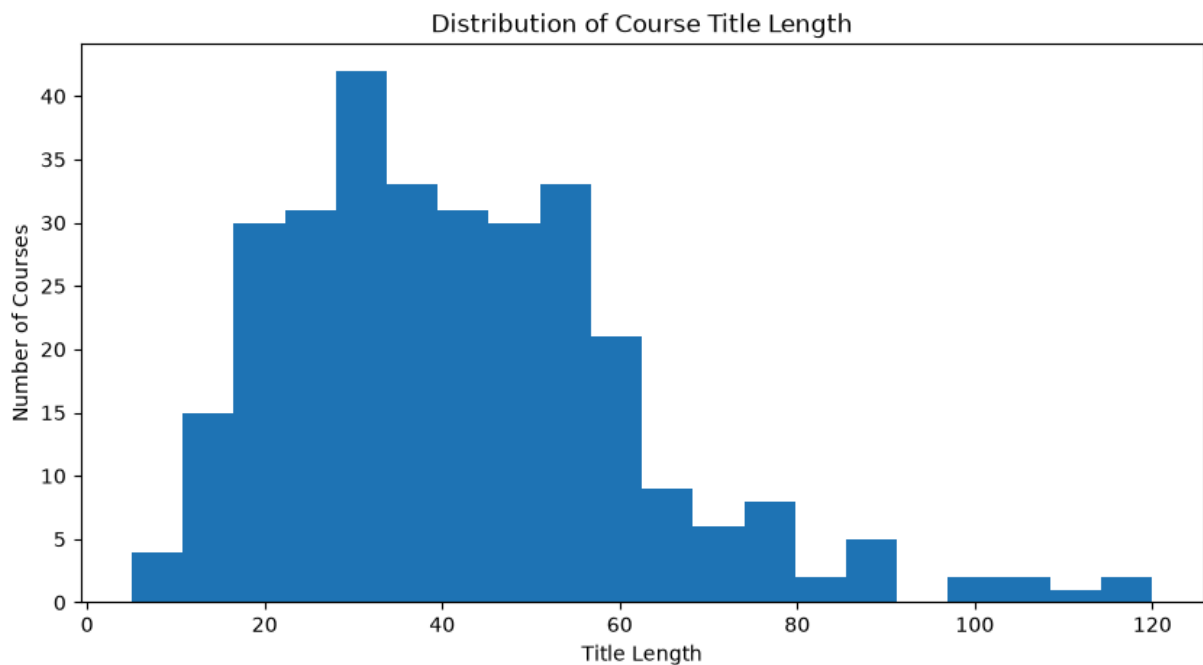
```
In [169... plt.figure(figsize=(10, 5))

plt.hist(df2_clean["title_length"], bins=20)

plt.title("Distribution of Course Title Length")
plt.xlabel("Title Length")
plt.ylabel("Number of Courses")

plt.savefig("course_title_length_distribution.png",
            dpi=300,
            bbox_inches="tight")

plt.show()
```



8.9 Course Text Missing-Value Analysis

The course title and description fields are checked for missing or empty text values.

This is important because missing textual information can affect NLP feature extraction and course similarity calculations.

```
In [170... # Missing values in title and description

print("Missing TITLE values:", df2_clean["TITLE"].isnull().sum())
print("Missing DESCRIPTION values:", df2_clean["DESCRIPTION"].isnull().sum())
```

```
Missing TITLE values: 0
Missing DESCRIPTION values: 0
```

```
In [171... # Empty text values

empty_titles = (df2_clean["TITLE"].fillna("").str.strip() == "").sum()
empty_descriptions = (df2_clean["DESCRIPTION"].fillna("").str.strip() == "").sum()
```

```
print("Empty TITLE values:", empty_titles)
print("Empty DESCRIPTION values:", empty_descriptions)
```

Empty TITLE values: 0
Empty DESCRIPTION values: 0

In [172...

```
# Text availability

text_availability = pd.DataFrame({
    "Text Field": ["TITLE", "DESCRIPTION"],
    "Available": [
        len(df2_clean) - empty_titles,
        len(df2_clean) - empty_descriptions
    ],
    "Missing_or_Empty": [
        empty_titles,
        empty_descriptions
    ]
})

display(text_availability)
```

	Text Field	Available	Missing_or_Empty
0	TITLE	307	0
1	DESCRIPTION	307	0

8.10 Course Title Word Analysis

Course titles are analyzed to identify the most frequently occurring words.

This provides an initial understanding of the main topics represented in the course catalog and prepares the data for later NLP-based recommendation techniques.

In [173...

```
# Combine all course titles

all_titles = " ".join(
    df2_clean["TITLE"].fillna("").astype(str)
)

print(all_titles[:1000])
```

robots are coming build iot apps with watson swift and node red accelerating deep learning with gpu consuming restful services using the reactive jax rs client analyzing big data in r using apache spark containerizing packaging and running a spring boot application cloud native security conference data security data science bootcamp with r for university professors learn how to use docker containers for iterative development scorm test 1 create your first mongodb database testing microservices with the arquillian managed container cloud pak for integration essentials watson analytics for social media data science bootcamp with python for university professors advance create a cryptocurrency trading algorithm in python data ai essentials securing java microservices with eclipse microprofile json web token microprofile jwt enabling distributed tracing in microservices with zipkin sql access for hadoop hybrid cloud conference ai pipelines lab dataops methodology data ai j

In [174...

```
# Word frequency in course titles

from collections import Counter
import re

words = re.findall(r"\b[a-zA-Z]+\b", all_titles.lower())

word_counts = Counter(words)

title_word_frequency = pd.DataFrame(
    word_counts.most_common(20),
    columns=["Word", "Frequency"]
)

display(title_word_frequency)
```

	Word	Frequency
0	data	89
1	with	81
2	and	58
3	using	46
4	in	42
5	for	40
6	to	39
7	learning	32
8	a	32
9	cloud	32
10	science	30
11	python	27
12	introduction	27
13	analysis	27
14	java	24
15	microservices	22
16	machine	21
17	the	17
18	r	17
19	big	15

In [175...

```
# Top 20 words in course titles

plt.figure(figsize=(10, 6))

sns.barplot(
    data=title_word_frequency,
    x="Frequency",
    y="Word"
)

plt.title("Top 20 Words in Course Titles")
plt.xlabel("Frequency")
plt.ylabel("Word")

plt.tight_layout()

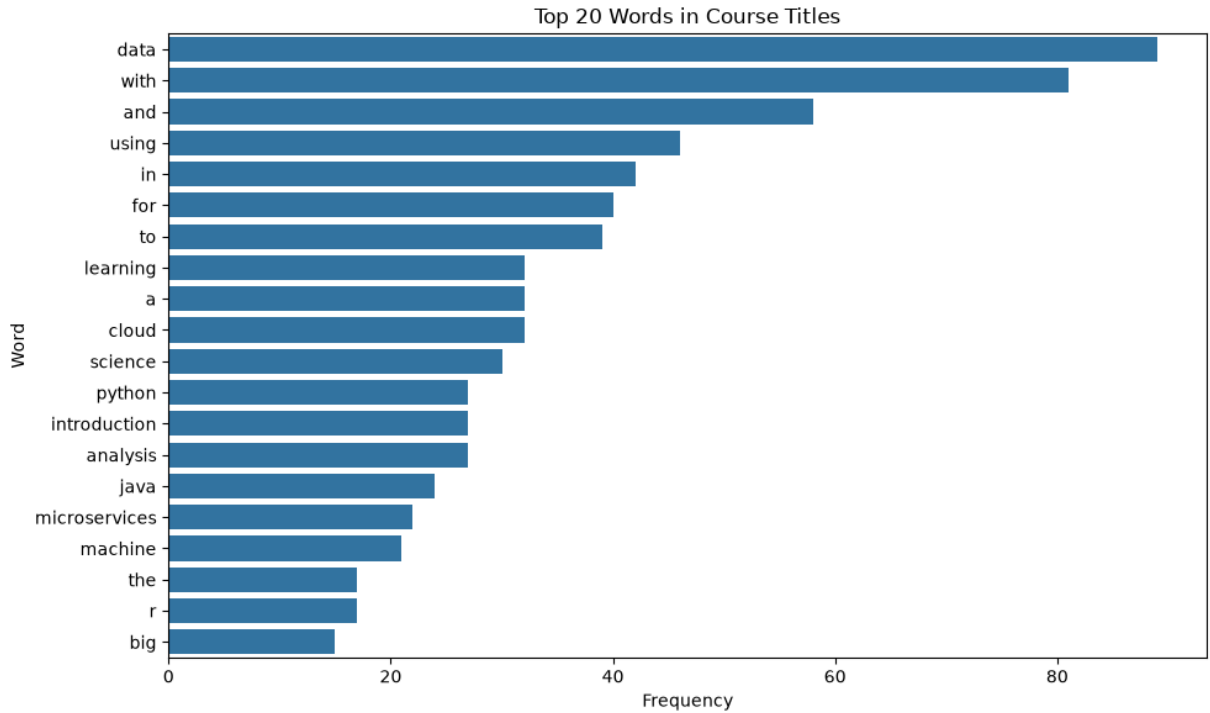
plt.savefig("top_20_course_title_words.png",
```

```

        dpi=300,
        bbox_inches="tight")

plt.show()

```



8.11 Course Title Text Preprocessing

Before applying NLP techniques such as Bag of Words and TF-IDF, course titles need basic text preprocessing.

The preprocessing includes converting text to lowercase and removing unnecessary characters. This creates a cleaner and more consistent text representation for feature extraction.

In [176...

```

import re

def preprocess_text(text):
    text = str(text).lower()
    text = re.sub(r"^[a-zA-Z0-9\s]", " ", text)
    text = re.sub(r"\s+", " ", text).strip()
    return text

df2_clean["processed_title"] = (
    df2_clean["TITLE"]
    .fillna("")
    .apply(preprocess_text)
)

display(
    df2_clean[
        "TITLE",
        "processed_title"
    ]
)

```

```
]].head(10)
)
```

	TITLE	processed_title
0	robots are coming build iot apps with watson ...	robots are coming build iot apps with watson S...
1	accelerating deep learning with gpu	accelerating deep learning with gpu
2	consuming restful services using the reactive ...	consuming restful services using the reactive ...
3	analyzing big data in r using apache spark	analyzing big data in r using apache spark
4	containerizing packaging and running a sprin...	containerizing packaging and running a spring ...
5	cloud native security conference data security	cloud native security conference data security
6	data science bootcamp with r for university pr...	data science bootcamp with r for university pr...
7	learn how to use docker containers for iterati...	learn how to use docker containers for iterati...
8	scorm test 1	scorm test 1
9	create your first mongodb database	create your first mongodb database

In [177...

```
# Compare original and processed title lengths

df2_clean["processed_title_length"] = (
    df2_clean["processed_title"].str.len()
)

display(
    df2_clean[[
        "TITLE",
        "processed_title",
        "processed_title_length"
    ]].head(10)
)
```

	TITLE	processed_title	processed_title_length
0	robots are coming build iot apps with watson ...	robots are coming build iot apps with watson s...	63
1	accelerating deep learning with gpu	accelerating deep learning with gpu	35
2	consuming restful services using the reactive ...	consuming restful services using the reactive ...	59
3	analyzing big data in r using apache spark	analyzing big data in r using apache spark	42
4	containerizing packaging and running a sprin...	containerizing packaging and running a spring ...	62
5	cloud native security conference data security	cloud native security conference data security	46
6	data science bootcamp with r for university pr...	data science bootcamp with r for university pr...	54
7	learn how to use docker containers for iterati...	learn how to use docker containers for iterati...	60
8	scorm test 1	scorm test 1	12
9	create your first mongodb database	create your first mongodb database	34

8.12 Stopword Analysis in Course Titles

Common words such as "the", "and", "of", and "in" can occur frequently in text without providing much information about the topic.

Stopword analysis helps identify these common terms before applying NLP techniques such as Bag of Words and TF-IDF. Removing such words can reduce unnecessary features and make the resulting text representation more focused on meaningful terms.

In [178...

```
from collections import Counter

# Common English stopwords
stop_words = {
    "a", "an", "the", "and", "or", "but", "if", "in", "on", "at",
    "to", "for", "of", "with", "by", "from", "is", "are", "was",
    "were", "be", "been", "this", "that", "these", "those", "as"
}

# Extract words from processed course titles
title_words = []

for title in df2_clean["processed_title"]:
    title_words.extend(title.split())
```

```

# Count all words
all_word_counts = Counter(title_words)

# Count stopwords
stopword_counts = Counter(
    word for word in title_words
    if word in stop_words
)

stopword_frequency = pd.DataFrame(
    stopword_counts.most_common(20),
    columns=["Stopword", "Frequency"]
)

display(stopword_frequency)

```

	Stopword	Frequency
0	with	81
1	and	58
2	in	42
3	for	40
4	to	39
5	a	32
6	the	17
7	of	12
8	on	8
9	an	6
10	by	4
11	from	3
12	as	2
13	are	1
14	or	1
15	at	1

In [179...

```

plt.figure(figsize=(10, 6))

sns.barplot(
    data=stopword_frequency,
    x="Frequency",
    y="Stopword"
)

plt.title("Common Stopwords in Course Titles")

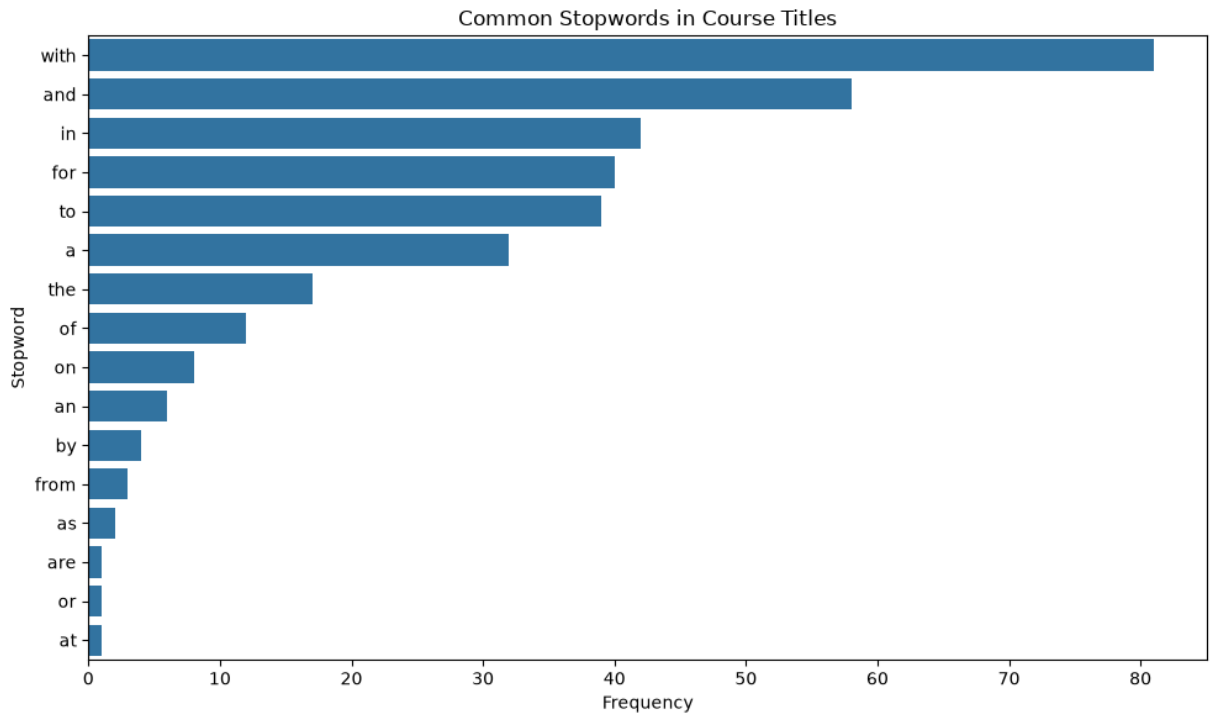
```

```
plt.xlabel("Frequency")
plt.ylabel("Stopword")

plt.tight_layout()

plt.savefig("course_title_stopwords.png",
            dpi=300,
            bbox_inches="tight")

plt.show()
```



9. Content-Based Recommendation

Content-based recommendation suggests courses by comparing their content and characteristics with the learner's interests.

In this project, course titles, descriptions, and genre information will be used to represent courses. Natural Language Processing (NLP) techniques will convert textual information into numerical features, allowing us to measure similarity between courses and identify relevant recommendations.

The content-based recommendation approach will use Bag of Words, TF-IDF, similarity measures, user profiles, clustering, and PCA to analyze the course catalog and generate personalized recommendations.

Contents

9.1 Bag of Words Feature Extraction

9.2 TF-IDF Feature Extraction

9.3 Course Similarity

9.4 Course Similarity Recommendations

9.5 User Profile Creation

9.6 User Profile and Course Similarity

9.7 User Profile Clustering

9.8 PCA-Based Course Analysis

9.1 Bag of Words Feature Extraction

Bag of Words (BoW) is a text representation technique that converts textual data into numerical features.

In this project, BoW will be applied to the processed course titles to create a document-term matrix. Each row represents a course, while each column represents a word from the vocabulary. The values indicate how frequently each word occurs in a course title.

The resulting numerical representation will be used in later stages to calculate course similarity and generate content-based recommendations.

In [180...

```
from sklearn.feature_extraction.text import CountVectorizer

# Create a lightweight Bag of Words model
bow_vectorizer = CountVectorizer(
    stop_words="english",
    max_features=1000
)

# Convert course titles into BoW features
bow_matrix = bow_vectorizer.fit_transform(
    df2_clean["processed_title"]
)

# Get vocabulary
bow_features = bow_vectorizer.get_feature_names_out()
```

```
print("Number of courses:", bow_matrix.shape[0])
print("Number of BoW features:", bow_matrix.shape[1])

print("\nFirst 15 vocabulary words:")
print(bow_features[:15])
```

Number of courses: 307

Number of BoW features: 479

First 15 vocabulary words:

```
['101' '201' '301' 'academic' 'accelerating' 'access' 'accessing'
 'acknowledging' 'action' 'actuarial' 'advance' 'advanced' 'agriculture'
 'ai' 'algolia']
```

In [181...

```
# Find the most frequent words

word_frequency = bow_matrix.sum(axis=0).A1

top_words = sorted(
    zip(bow_features, word_frequency),
    key=lambda x: x[1],
    reverse=True
)[:10]

print("Top 10 words in course titles:")

for word, frequency in top_words:
    print(word, ":", frequency)
```

Top 10 words in course titles:

```
data : 89
using : 46
cloud : 32
learning : 32
science : 30
analysis : 27
introduction : 27
python : 27
java : 24
microservices : 22
```

In [182...

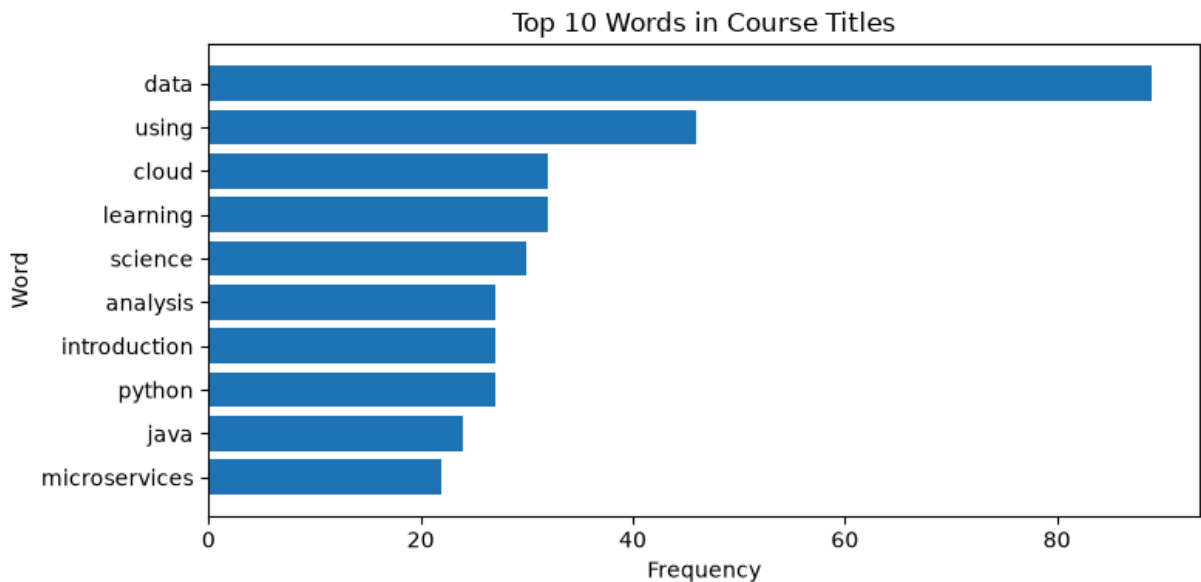
```
# Visualize the top 10 BoW words

words = [x[0] for x in top_words]
frequencies = [x[1] for x in top_words]

plt.figure(figsize=(8, 4))
plt.barh(words[::-1], frequencies[::-1])

plt.title("Top 10 Words in Course Titles")
plt.xlabel("Frequency")
plt.ylabel("Word")

plt.tight_layout()
plt.savefig("top_10_bow_words.png", dpi=300, bbox_inches="tight")
plt.show()
```



9.2 TF-IDF Feature Extraction

TF-IDF (Term Frequency–Inverse Document Frequency) converts course text into numerical features while giving greater importance to words that are useful for distinguishing one course from another.

Unlike Bag of Words, TF-IDF reduces the importance of words that appear frequently across many course titles.

A lightweight TF-IDF representation will be created using a maximum of 1,000 features. This representation will later be used for course similarity and recommendations.

In [183... `from sklearn.feature_extraction.text import TfidfVectorizer`

```
tfidf_vectorizer = TfidfVectorizer(  
    stop_words="english",  
    max_features=1000  
)  
  
tfidf_matrix = tfidf_vectorizer.fit_transform(  
    df2_clean["processed_title"]  
)  
  
tfidf_features = tfidf_vectorizer.get_feature_names_out()  
  
print("Courses:", tfidf_matrix.shape[0])  
print("TF-IDF features:", tfidf_matrix.shape[1])
```

Courses: 307

TF-IDF features: 479

In [184... `# Find the most important words`

```
tfidf_scores = tfidf_matrix.sum(axis=0).A1
```

```

top_tfidf = sorted(
    zip(tfidf_features, tfidf_scores),
    key=lambda x: x[1],
    reverse=True
)[:10]

print("Top 10 TF-IDF words:")

for word, score in top_tfidf:
    print(word, ":", round(score, 3))

```

```

Top 10 TF-IDF words:
data : 24.144
learning : 12.862
python : 12.178
introduction : 12.133
science : 11.845
using : 11.556
analysis : 11.541
cloud : 11.059
101 : 9.872
machine : 9.792

```

In [186... *# Display a small sample of the TF-IDF matrix*

```

tfidf_sample = tfidf_matrix[:5, :10].toarray()

print("First 5 courses x first 10 TF-IDF features:")
print(tfidf_sample)

```

```

First 5 courses x first 10 TF-IDF features:
[[0.      0.      0.      0.      0.      0.
  0.      0.      0.      0.      ]
 [0.      0.      0.      0.      0.56660893 0.
  0.      0.      0.      0.      ]
 [0.      0.      0.      0.      0.      0.
  0.      0.      0.      0.      ]
 [0.      0.      0.      0.      0.      0.
  0.      0.      0.      0.      ]
 [0.      0.      0.      0.      0.      0.
  0.      0.      0.      0.      ]]

```

In [187... `tfidf_sample_df = pd.DataFrame(`
`tfidf_matrix[:5, :10].toarray(),`
`columns=tfidf_features[:10]`
`)`
`print("TF-IDF Matrix Sample:")`
`print(tfidf_sample_df)`

TF-IDF Matrix Sample:

	101	201	301	academic	accelerating	access	accessing	acknowledging	\
0	0.0	0.0	0.0	0.0	0.000000	0.0	0.0	0.0	
1	0.0	0.0	0.0	0.0	0.566609	0.0	0.0	0.0	
2	0.0	0.0	0.0	0.0	0.000000	0.0	0.0	0.0	
3	0.0	0.0	0.0	0.0	0.000000	0.0	0.0	0.0	
4	0.0	0.0	0.0	0.0	0.000000	0.0	0.0	0.0	

	action	actuarial
0	0.0	0.0
1	0.0	0.0
2	0.0	0.0
3	0.0	0.0
4	0.0	0.0

In [185...

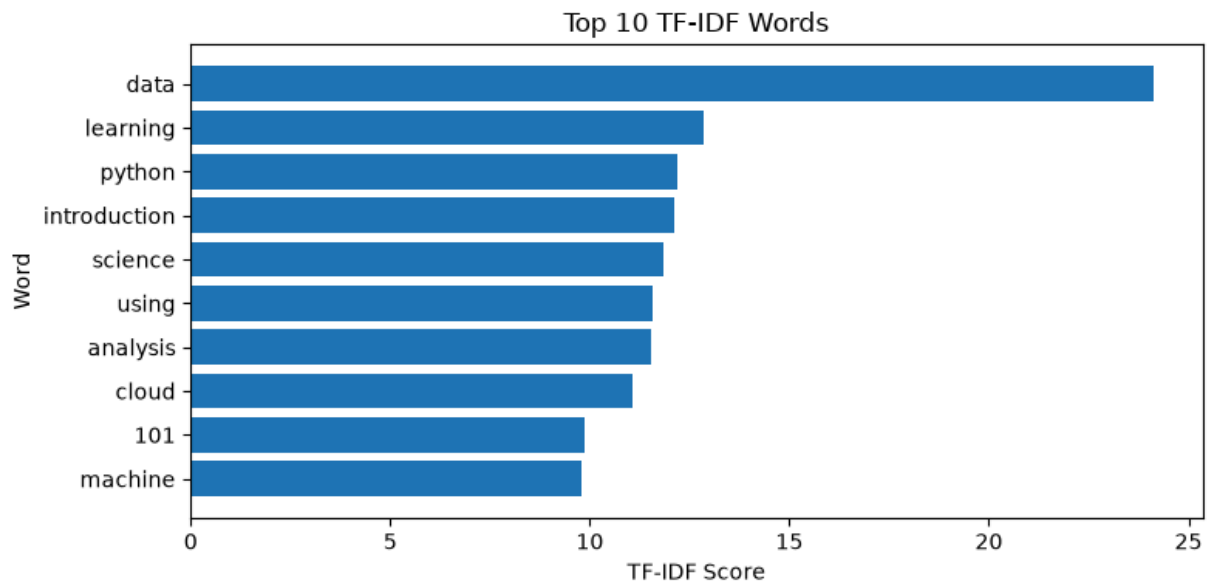
```
# Visualize top TF-IDF words

words = [x[0] for x in top_tfidf]
scores = [x[1] for x in top_tfidf]

plt.figure(figsize=(8, 4))
plt.barh(words[::-1], scores[::-1])

plt.title("Top 10 TF-IDF Words")
plt.xlabel("TF-IDF Score")
plt.ylabel("Word")

plt.tight_layout()
plt.savefig("top_10_tfidf_words.png", dpi=300, bbox_inches="tight")
plt.show()
```



9.3 Course Similarity

Course similarity measures how closely two courses are related based on their textual content.

In this project, cosine similarity is used to compare the TF-IDF representation of courses. A higher cosine similarity score indicates that two courses have more similar content.

To keep the computation lightweight, similarity will be calculated only when recommendations are requested rather than creating a large similarity matrix for all courses.

```
In [188... from sklearn.metrics.pairwise import cosine_similarity

def find_similar_courses(course_index, n=5):

    # Compare one course with all courses
    scores = cosine_similarity(
        tfidf_matrix[course_index],
        tfidf_matrix
    ).flatten()

    # Get the most similar courses
    similar_indices = scores.argsort()[::-1]

    # Remove the selected course
    similar_indices = similar_indices[
        similar_indices != course_index
   ][:n]

    return similar_indices, scores[similar_indices]
```

```
In [189... # Test similarity for the first course

course_index = 0

similar_indices, similarity_scores = find_similar_courses(
    course_index,
    n=5
)

print("Selected course:")
print(df2_clean.iloc[course_index]["TITLE"])

print("\nSimilar courses:")

for index, score in zip(similar_indices, similarity_scores):
    print(
        df2_clean.iloc[index]["TITLE"],
        "->",
        round(score, 3)
    )
```

Selected course:
robots are coming build iot apps with watson swift and node red

Similar courses:
build swift mobile apps with watson ai services -> 0.461
node red basics to bots -> 0.347
how to build watson ai and swift apis and make money -> 0.304
build chatbots with watson assistant -> 0.243
build an iot blockchain network for a supply chain -> 0.231

9.4 Course Similarity Recommendations

The course similarity scores can be used to recommend courses that are most similar to a selected course.

For a given course, the system identifies the courses with the highest cosine similarity scores and presents them as recommendations. The selected course itself is excluded from the recommendation list.

This provides a simple content-based recommendation approach based on course title information.

```
In [191... # Recommend similar courses

selected_index = 0

similar_indices, similarity_scores = find_similar_courses(
    selected_index,
    n=5
)

print("Selected Course:")
print(df2_clean.iloc[selected_index]["TITLE"])

print("\nRecommended Courses:")

for index, score in zip(similar_indices, similarity_scores):
    print(
        f"{df2_clean.iloc[index]['TITLE']} "
        f"(Similarity: {score:.3f})"
    )
```

Selected Course:
robots are coming build iot apps with watson swift and node red

Recommended Courses:
build swift mobile apps with watson ai services (Similarity: 0.461)
node red basics to bots (Similarity: 0.347)
how to build watson ai and swift apis and make money (Similarity: 0.304)
build chatbots with watson assistant (Similarity: 0.243)
build an iot blockchain network for a supply chain (Similarity: 0.231)

In [192...

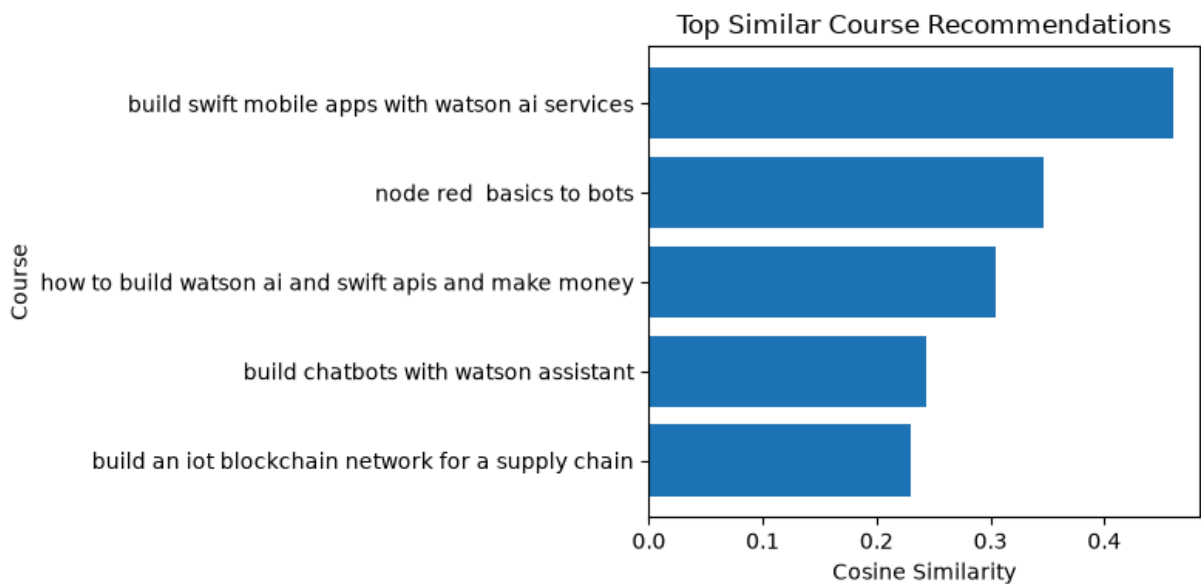
```
# Small graph of recommendations

recommendation_titles = [
    df2_clean.iloc[index]["TITLE"]
    for index in similar_indices
]

plt.figure(figsize=(8, 4))
plt.barh(
    recommendation_titles[::-1],
    similarity_scores[::-1]
)

plt.title("Top Similar Course Recommendations")
plt.xlabel("Cosine Similarity")
plt.ylabel("Course")

plt.tight_layout()
plt.savefig(
    "top_similar_course_recommendations.png",
    dpi=300,
    bbox_inches="tight"
)
plt.show()
```



9.5 User Profile Creation

A user profile represents the interests of a learner based on the courses they have interacted with.

In this project, the genres of courses interacted with by each learner are used to create a simple user profile. The profile is based on the learner's interaction history rather than creating a large text-based user vector.

This lightweight approach allows us to understand learner preferences and use them for personalized course recommendations.

```
In [193... # Create simple user profiles using course genres

genre_columns = [
    "Database", "Python", "CloudComputing", "DataAnalysis",
    "Containers", "MachineLearning", "ComputerVision",
    "DataScience", "BigData", "Chatbot", "R",
    "BackendDev", "FrontendDev", "Blockchain"
]

# Combine user interactions with course genres
user_genres = df3_clean.merge(
    df1_clean[["COURSE_ID"] + genre_columns],
    left_on="item",
    right_on="COURSE_ID",
    how="inner"
)

# Calculate average genre preference for each user
user_profiles = user_genres.groupby("user")[genre_columns].mean()

print("Number of users:", len(user_profiles))
print("Number of genre features:", len(genre_columns))

print("\nFirst user profile:")
print(user_profiles.iloc[0])
```

Number of users: 33901

Number of genre features: 14

First user profile:

Database	0.327869
Python	0.081967
CloudComputing	0.032787
DataAnalysis	0.262295
Containers	0.016393
MachineLearning	0.196721
ComputerVision	0.000000
DataScience	0.180328
BigData	0.262295
Chatbot	0.016393
R	0.098361
BackendDev	0.196721
FrontendDev	0.049180
Blockchain	0.032787

Name: 2, dtype: float64

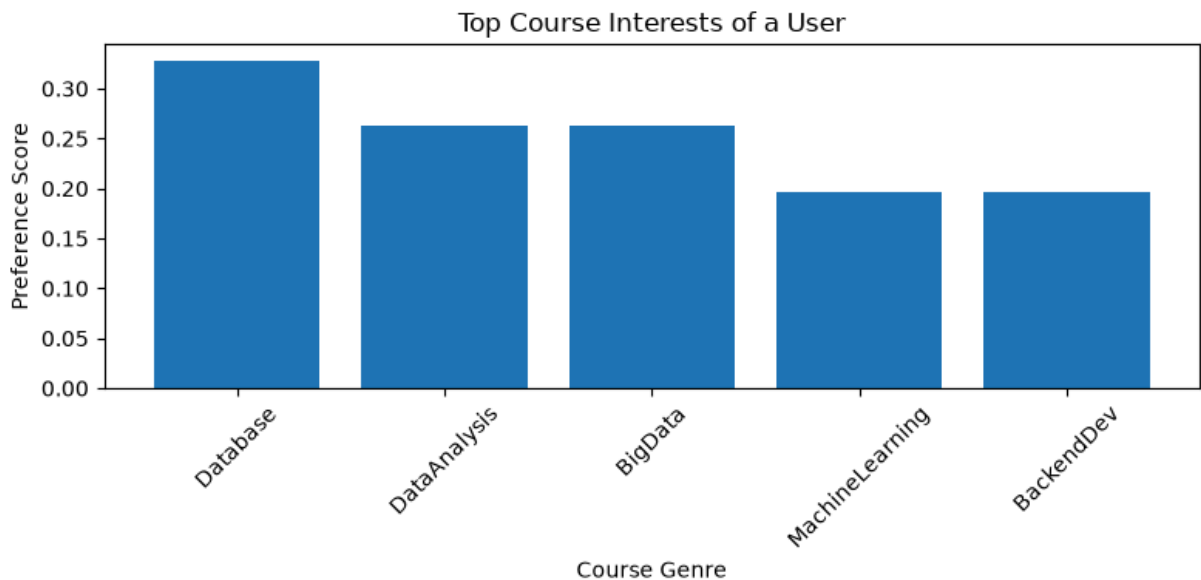
```
In [194... # Display the strongest interests of the first user

first_user = user_profiles.iloc[0].sort_values(
    ascending=False
).head(5)
```

```
plt.figure(figsize=(8, 4))
plt.bar(
    first_user.index,
    first_user.values
)

plt.title("Top Course Interests of a User")
plt.xlabel("Course Genre")
plt.ylabel("Preference Score")
plt.xticks(rotation=45)

plt.tight_layout()
plt.savefig(
    "user_profile_interests.png",
    dpi=300,
    bbox_inches="tight"
)
plt.show()
```



9.6 User Profile and Course Similarity

The user profiles created from course genres can be compared with the genre characteristics of courses.

For each learner, the system identifies courses whose genre composition is closest to the learner's interests. Cosine similarity is used to measure the similarity between the learner profile and each course profile.

Only a single selected user is evaluated at a time to keep the computation lightweight.

```
In [195... from sklearn.metrics.pairwise import cosine_similarity

# Create a course genre matrix
course_profiles = (
```

```

    df1_clean
    .set_index("COURSE_ID")[genre_columns]
)

# Select one user
selected_user = user_profiles.index[0]

# Get the user's profile
user_vector = user_profiles.loc[
    selected_user
].values.reshape(1, -1)

# Calculate similarity with all courses
scores = cosine_similarity(
    user_vector,
    course_profiles.values
).flatten()

# Get top 5 courses
top_indices = scores.argsort()[::-1][:5]

print("Selected User:", selected_user)
print("\nRecommended courses based on user interests:")

for index in top_indices:
    course_id = course_profiles.index[index]

    title = df2_clean.loc[
        df2_clean["COURSE_ID"] == course_id,
        "TITLE"
    ].iloc[0]

    print(f"{title} (Similarity: {scores[index]:.3f})")

```

Selected User: 2

Recommended courses based on user interests:
 getting started with the data apache spark makers build (Similarity: 0.802)
 foundations for big data analysis with sql (Similarity: 0.802)
 analyzing big data with sql (Similarity: 0.802)
 spark overview for scala analytics (Similarity: 0.802)
 spark fundamentals ii (Similarity: 0.802)

In [196... *# Small recommendation graph*

```

top_scores = scores[top_indices]

plt.figure(figsize=(8, 4))

plt.barh(
    range(5),
    top_scores[::-1]
)

plt.title("Top Courses Matching User Interests")
plt.xlabel("Cosine Similarity")

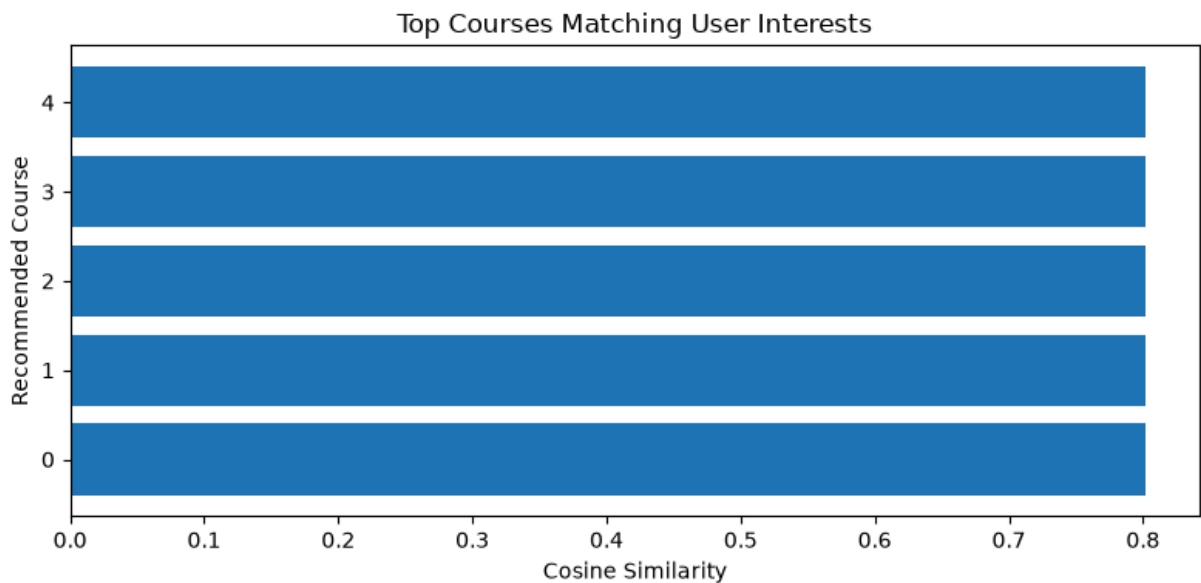
```

```
plt.ylabel("Recommended Course")

plt.tight_layout()

plt.savefig(
    "user_course_similarity.png",
    dpi=300,
    bbox_inches="tight"
)

plt.show()
```



9.7 User Profile Clustering

User profile clustering groups learners with similar course interests.

K-Means clustering is used to identify a small number of learner groups based on their genre preferences. This can help the recommendation system understand different types of learners without performing expensive calculations on the complete interaction dataset.

A small number of clusters will be tested, and the silhouette score will be used to select the better clustering configuration.

```
In [197... from sklearn.cluster import KMeans
from sklearn.metrics import silhouette_score

# Use the user profile genre features
X_users = user_profiles.values

# Test a small number of clusters
results = []

for k in range(2, 5):
    model = KMeans(
```

```

        n_clusters=k,
        random_state=RANDOM_STATE,
        n_init=10
    )

    labels = model.fit_predict(X_users)

    score = silhouette_score(
        X_users,
        labels
    )

    results.append((k, score))

print("Clustering results:")

for k, score in results:
    print(f"Clusters: {k} | Silhouette Score: {score:.3f}")

```

Clustering results:
Clusters: 2 | Silhouette Score: 0.213
Clusters: 3 | Silhouette Score: 0.274
Clusters: 4 | Silhouette Score: 0.322

In [198... *# Select the best cluster count*

```

best_k = max(
    results,
    key=lambda x: x[1]
)[0]

user_cluster_model = KMeans(
    n_clusters=best_k,
    random_state=RANDOM_STATE,
    n_init=10
)

user_clusters = user_cluster_model.fit_predict(
    X_users
)

print("Selected number of clusters:", best_k)

```

Selected number of clusters: 4

In [199... *# Small cluster-size graph*

```

cluster_counts = pd.Series(
    user_clusters
).value_counts().sort_index()

plt.figure(figsize=(6, 4))

plt.bar(
    cluster_counts.index.astype(str),
    cluster_counts.values
)

```

```
plt.title("Learner Distribution Across Clusters")
plt.xlabel("Cluster")
plt.ylabel("Number of Users")

plt.tight_layout()

plt.savefig(
    "user_profile_clusters.png",
    dpi=300,
    bbox_inches="tight"
)

plt.show()
```



9.8 PCA-Based Course Analysis

Principal Component Analysis (PCA) is a dimensionality-reduction technique that transforms a large number of features into a smaller set of principal components.

In this project, PCA is applied to the course genre features to visualize the overall structure of the course catalog in two dimensions. This lightweight approach avoids converting the large TF-IDF matrix into a dense matrix.

The first two principal components will be used to identify broad patterns in the course dataset.

In [200... `from sklearn.decomposition import PCA`

```

# Use course genre features for PCA
X_courses = df1_clean[genre_columns].values

# Reduce to two principal components
pca = PCA(n_components=2)

course_pca = pca.fit_transform(X_courses)

print("Explained variance:")
print("PC1:", round(pca.explained_variance_ratio_[0], 3))
print("PC2:", round(pca.explained_variance_ratio_[1], 3))
print(
    "Total:",
    round(pca.explained_variance_ratio_.sum(), 3)
)

```

Explained variance:

PC1: 0.204

PC2: 0.19

Total: 0.394

In [201...

```

# Small PCA visualization

plt.figure(figsize=(7, 5))

plt.scatter(
    course_pca[:, 0],
    course_pca[:, 1],
    s=15,
    alpha=0.5
)

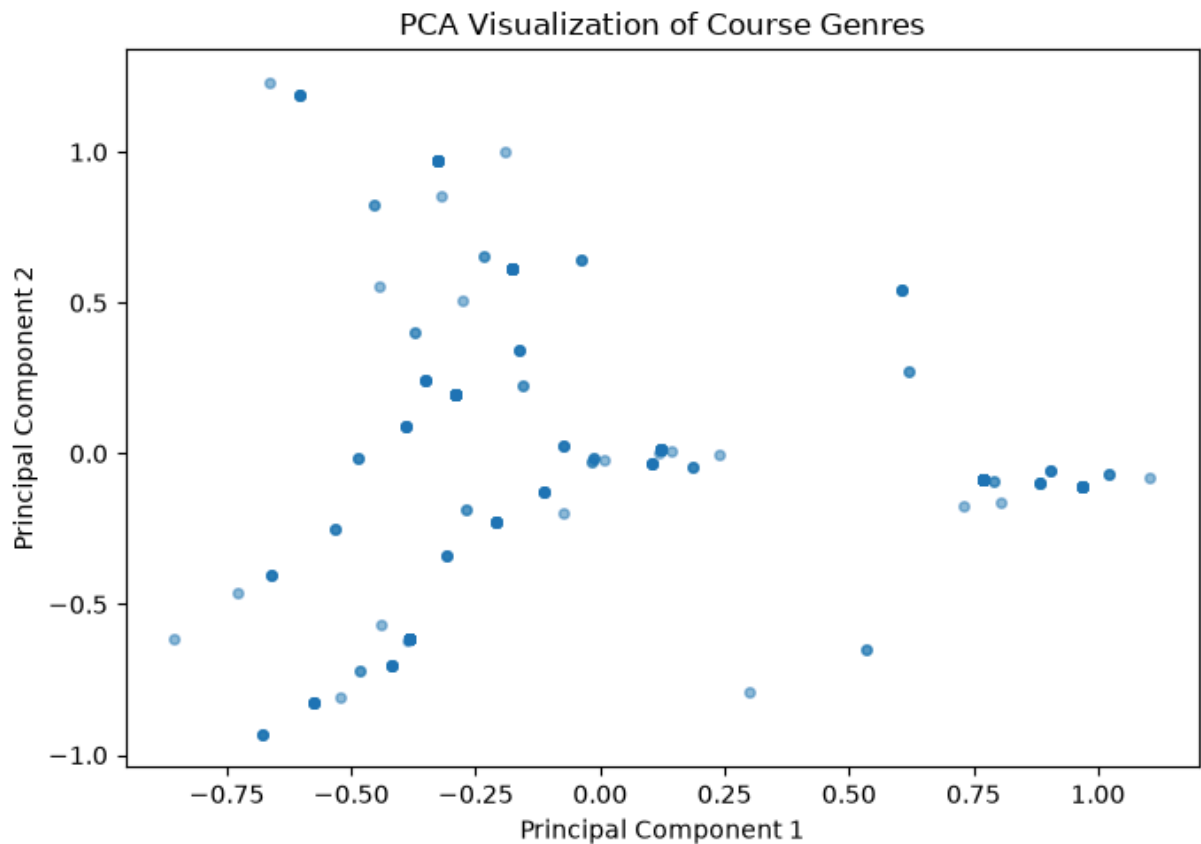
plt.title("PCA Visualization of Course Genres")
plt.xlabel("Principal Component 1")
plt.ylabel("Principal Component 2")

plt.tight_layout()

plt.savefig(
    "pca_course_genres.png",
    dpi=300,
    bbox_inches="tight"
)

plt.show()

```



10. Collaborative Filtering

Collaborative filtering recommends courses based on patterns in learner-course interactions.

Instead of relying only on course content, this approach uses the behavior of learners to identify courses that may be relevant to them. Learners with similar interaction patterns can be used to generate recommendations.

In this section, lightweight versions of KNN, NMF, and neural-network-based collaborative filtering will be explored. The models will be designed to avoid large user-course calculations so that they can run efficiently on the available system.

10.1 User-Course Rating Matrix

10.2 KNN Collaborative Filtering

10.3 NMF Collaborative Filtering

10.4 Neural Network Collaborative Filtering

10.5 Additional Collaborative Filtering Models

10.1 User-Course Rating Matrix

A user-course rating matrix represents the interactions between learners and courses.

Each row represents a learner, each column represents a course, and the values represent the ratings given by learners.

This matrix provides the basic structure required for collaborative filtering. To keep the computation lightweight, only a small sample of users and courses will be used for visualization, while the complete interaction data remains available for later modeling.

In [202...

```
# Create the user-course rating matrix

user_course_matrix = df3_clean.pivot_table(
    index="user",
    columns="item",
    values="rating",
    aggfunc="mean",
    fill_value=0
)

print("Users:", user_course_matrix.shape[0])
print("Courses:", user_course_matrix.shape[1])
```

Users: 33901
Courses: 126

In [203...

```
# Display only a small sample

print("Sample of the user-course rating matrix:")
print(user_course_matrix.iloc[:5, :10])
```

Sample of the user-course rating matrix:

item	AI0111EN	BC0101EN	BC0201EN	BC0202EN	BD0101EN	BD0111EN	BD0115EN	\
------	----------	----------	----------	----------	----------	----------	----------	---

user								
2	0.0	4.0	0.0	0.0	5.0	4.0	0.0	
4	0.0	0.0	0.0	0.0	5.0	3.0	4.0	
5	3.0	5.0	5.0	0.0	4.0	0.0	0.0	
7	0.0	0.0	0.0	0.0	0.0	0.0	0.0	
8	0.0	0.0	0.0	0.0	0.0	3.0	0.0	

item	BD0121EN	BD0123EN	BD0131EN
------	----------	----------	----------

user			
2	5.0	3.0	3.0
4	5.0	3.0	4.0
5	0.0	3.0	0.0
7	0.0	0.0	0.0
8	0.0	0.0	0.0

In [204...

```
# Calculate matrix density
```

```

total_cells = (
    user_course_matrix.shape[0] *
    user_course_matrix.shape[1]
)

observed_cells = (
    user_course_matrix != 0
).sum().sum()

density = (
    observed_cells / total_cells
) * 100

print("Matrix density:", round(density, 4), "%")

```

Matrix density: 5.4619 %

In [205...

```

# Small visualization of a sample of the matrix

plt.figure(figsize=(7, 4))

plt.imshow(
    user_course_matrix.iloc[:15, :15],
    aspect="auto"
)

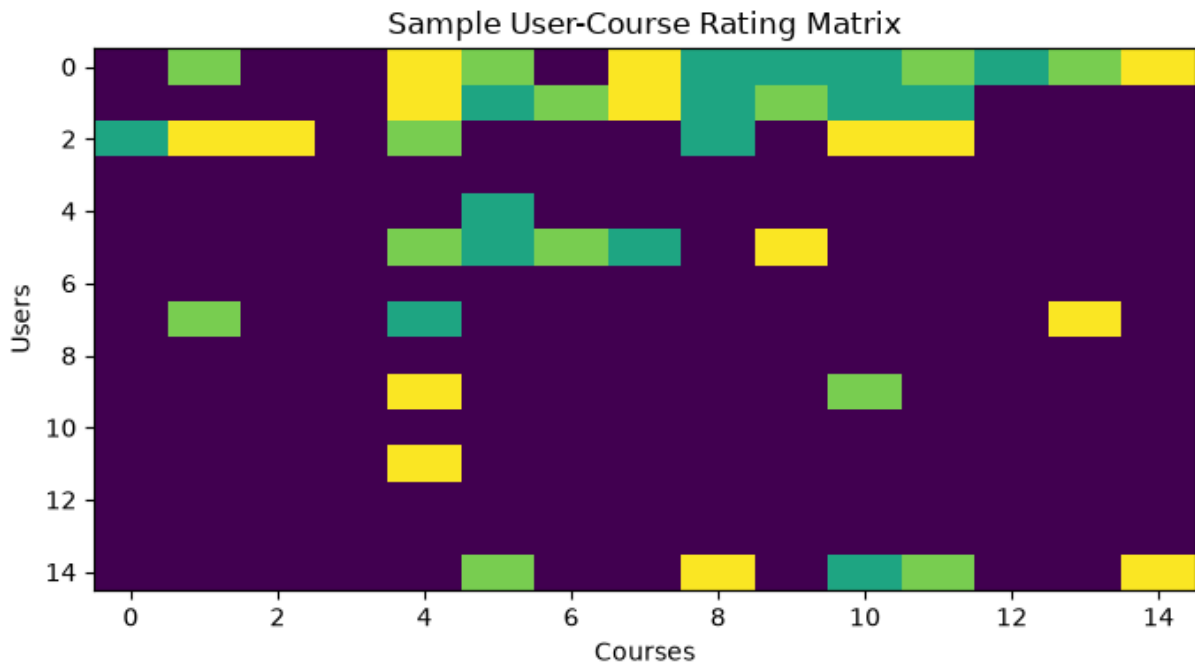
plt.title("Sample User-Course Rating Matrix")
plt.xlabel("Courses")
plt.ylabel("Users")

plt.tight_layout()

plt.savefig(
    "sample_user_course_matrix.png",
    dpi=300,
    bbox_inches="tight"
)

plt.show()

```



10.2 KNN Collaborative Filtering

K-Nearest Neighbors (KNN) recommends courses by finding learners with similar rating patterns.

In this approach, the rating vectors of learners are compared using cosine similarity. Courses that similar learners have rated highly can then be considered as potential recommendations.

To keep the computation lightweight, KNN is applied to a small sample of users rather than performing expensive calculations across the complete user population.

```
In [206... from sklearn.neighbors import NearestNeighbors

# Use a small sample of users
sample_users = user_course_matrix.iloc[:300]

# KNN model using cosine distance
knn_model = NearestNeighbors(
    n_neighbors=6,
    metric="cosine",
    algorithm="brute"
)

knn_model.fit(sample_users)

# Select one user
selected_user_index = 0

distances, indices = knn_model.kneighbors(
    sample_users.iloc[
```

```

        selected_user_index:selected_user_index + 1
    ]
)

print("Selected User:")
print(sample_users.index[selected_user_index])

print("\nSimilar Users:")

for i, distance in zip(indices[0][1:], distances[0][1:]):
    print(
        sample_users.index[i],
        "-> Similarity:",
        round(1 - distance, 3)
    )

```

Selected User:

2

Similar Users:

50831 -> Similarity: 0.69

61 -> Similarity: 0.619

21577 -> Similarity: 0.598

4 -> Similarity: 0.597

34074 -> Similarity: 0.579

In [207...

```

# Get similar users
similar_user_indices = indices[0][1:]

# Ratings of similar users
similar_user_ratings = sample_users.iloc[
    similar_user_indices
]

# Average their course ratings
course_scores = similar_user_ratings.mean()

# Exclude courses already rated by the selected user
selected_user_ratings = sample_users.iloc[
    selected_user_index
]

unrated_courses = selected_user_ratings[
    selected_user_ratings == 0
].index

recommendations = course_scores[
    unrated_courses
].sort_values(
    ascending=False
).head(5)

print("Recommended Courses:")

for course_id, score in recommendations.items():
    title = df2_clean.loc[

```

```

        df2_clean["COURSE_ID"] == course_id,
        "TITLE"
    ]

    if len(title) > 0:
        print(
            f"{title.iloc[0]} "
            f"(Score: {score:.2f})"
        )

```

Recommended Courses:

mapreduce and yarn (Score: 3.80)
 introduction to data science (Score: 3.80)
 nosql and dbaas 101 (Score: 2.60)
 data science hands on with open source tools (Score: 2.00)
 watson analytics for social media (Score: 1.60)

In [208... *# Average rating of recommended courses*

```

if len(recommendations) > 0:
    print(
        "Average recommendation score:",
        round(recommendations.mean(), 2)
    )

```

Average recommendation score: 2.76

10.3 NMF Collaborative Filtering

Non-Negative Matrix Factorization (NMF) decomposes the user-course rating matrix into smaller non-negative matrices.

The resulting latent factors represent hidden patterns in learner preferences and course characteristics. These factors can be used to reconstruct missing ratings and generate recommendations.

NMF is suitable for this project because course ratings are non-negative. To keep the computation lightweight, a limited number of components will be used.

In [209... **from** sklearn.decomposition **import** NMF

```

# Use a small sample of the rating matrix
nmf_data = user_course_matrix.iloc[:300, :300]

# Create NMF model
nmf_model = NMF(
    n_components=5,
    init="random",
    random_state=RANDOM_STATE,
    max_iter=100
)

# Learn latent factors
user_factors = nmf_model.fit_transform(nmf_data)

```

```

course_factors = nmf_model.components_

print("User factor shape:", user_factors.shape)
print("Course factor shape:", course_factors.shape)
print("Reconstruction error:", round(nmf_model.reconstruction_err_, 3))

```

User factor shape: (300, 5)
Course factor shape: (5, 126)
Reconstruction error: 122.266

```

In [210... # Reconstruct the rating matrix
predicted_ratings = np.dot(
    user_factors,
    course_factors
)

# Recommendations for the first user
user_ratings = nmf_data.iloc[0].values

unrated = user_ratings == 0

scores = predicted_ratings[0].copy()
scores[~unrated] = -1

top_courses = np.argsort(scores)[::-1][:5]

print("Top NMF recommended course IDs:")

for idx in top_courses:
    print(
        nmf_data.columns[idx],
        "-> Predicted score:",
        round(scores[idx], 2)
    )

```

Top NMF recommended course IDs:
BD0115EN -> Predicted score: 3.49
DB0151EN -> Predicted score: 2.91
BC0201EN -> Predicted score: 2.28
ML0151EN -> Predicted score: 2.13
DB0113EN -> Predicted score: 2.03

10.4 Neural Network Collaborative Filtering

Neural networks can learn nonlinear relationships between learners and courses.

In this approach, user IDs and course IDs are represented using embedding layers. These embeddings are combined and passed through a small neural network to predict the learner's rating for a course.

A lightweight architecture is used to keep training fast while demonstrating a deep-learning-based collaborative filtering approach.

In [211...

```
import tensorflow as tf
from tensorflow.keras import layers, Model

# Use a small sample for lightweight training
nn_data = df3_clean.head(5000).copy()

# Convert user and course IDs to integer indices
user_ids = nn_data["user"].astype("category")
course_ids = nn_data["item"].astype("category")

user_index = user_ids.cat.codes.values
course_index = course_ids.cat.codes.values

n_users = user_ids.cat.categories.size
n_courses = course_ids.cat.categories.size

# User and course inputs
user_input = layers.Input(shape=(1,))
course_input = layers.Input(shape=(1,))

# Embeddings
user_embedding = layers.Embedding(
    n_users, 8
)(user_input)

course_embedding = layers.Embedding(
    n_courses, 8
)(course_input)

# Flatten embeddings
user_vector = layers.Flatten()(user_embedding)
course_vector = layers.Flatten()(course_embedding)

# Combine
combined = layers.Concatenate()([
    user_vector,
    course_vector
])

# Small neural network
hidden = layers.Dense(16, activation="relu")(combined)
output = layers.Dense(1)(hidden)

model = Model(
    inputs=[user_input, course_input],
    outputs=output
)

model.compile(
    optimizer="adam",
    loss="mse"
)

model.fit(
    [user_index, course_index],
```

```

nn_data["rating"].values,
epochs=3,
batch_size=64,
verbose=1
)

print("Neural network collaborative filtering model trained.")

```

```

Epoch 1/3
79/79 ————— 3s 18ms/step - loss: 14.9531
Epoch 2/3
79/79 ————— 0s 4ms/step - loss: 8.4099
Epoch 3/3
79/79 ————— 0s 4ms/step - loss: 2.2154
Neural network collaborative filtering model trained.

```

In [212... *# Evaluate training error*

```

loss = model.evaluate(
    [user_index, course_index],
    nn_data["rating"].values,
    verbose=0
)

print("Neural Network MSE:", round(loss, 4))

```

Neural Network MSE: 1.1977

In [213... `history = model.fit(`

```

    [user_index, course_index],
    nn_data["rating"].values,
    epochs=3,
    batch_size=64,
    verbose=1
)

```

```

Epoch 1/3
79/79 ————— 0s 5ms/step - loss: 0.9823
Epoch 2/3
79/79 ————— 1s 6ms/step - loss: 0.6465
Epoch 3/3
79/79 ————— 0s 5ms/step - loss: 0.4224

```

In [214... *# Neural network training loss*

```

plt.figure(figsize=(7, 4))

plt.plot(
    history.history["loss"],
    marker="o"
)

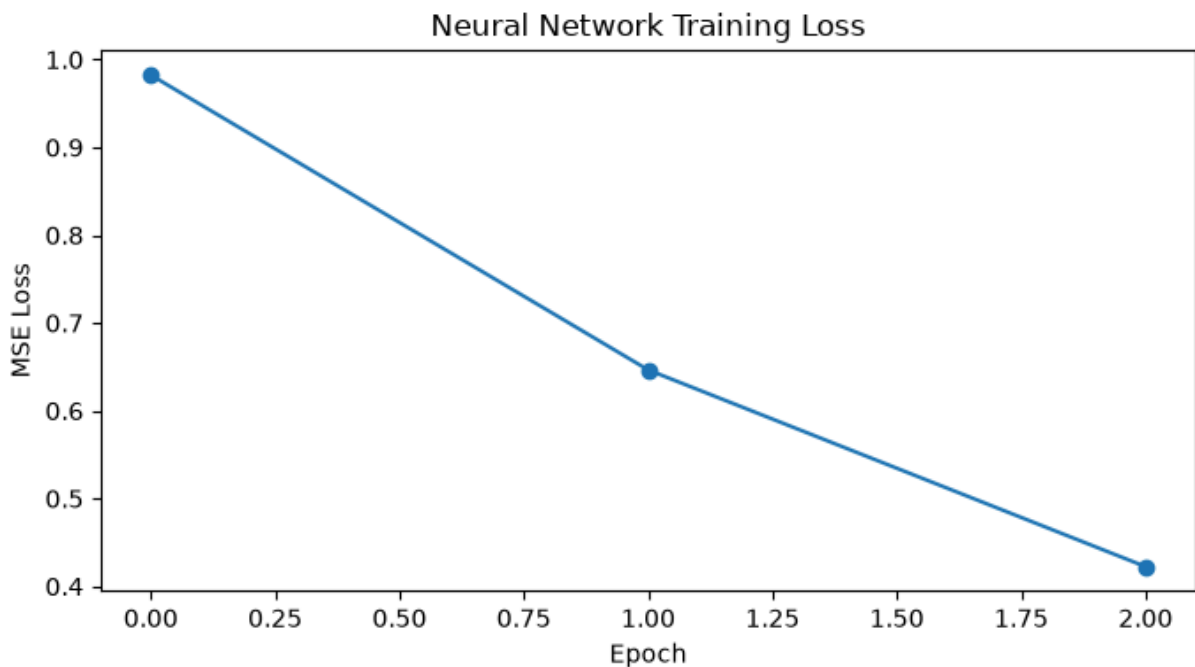
plt.title("Neural Network Training Loss")
plt.xlabel("Epoch")
plt.ylabel("MSE Loss")

plt.tight_layout()

```

```
plt.savefig(
    "neural_network_training_loss.png",
    dpi=300,
    bbox_inches="tight"
)

plt.show()
```



10.5 Additional Collaborative Filtering Models

Additional machine learning models can also be used for collaborative filtering by learning patterns between learners and courses.

In this project, lightweight versions of Linear Regression, Logistic Regression, and Random Forest are explored as additional approaches.

These models provide alternative perspectives for predicting learner-course preferences and allow broader comparison of machine learning approaches.

In [215...

```
from sklearn.linear_model import LinearRegression
from sklearn.model_selection import train_test_split

# Use a small sample
lr_data = df3_clean.head(5000).copy()

X = pd.get_dummies(
    lr_data[["user", "item"]]
).astype(float)

y = lr_data["rating"]
```

```

X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.2,
    random_state=RANDOM_STATE
)

lr_model = LinearRegression()
lr_model.fit(X_train, y_train)

lr_score = lr_model.score(X_test, y_test)

print("Linear Regression R²:", round(lr_score, 3))

```

Linear Regression R²: -0.003

In [216...

```

from sklearn.linear_model import LogisticRegression

# Convert ratings into positive / lower-rating classes
y_class = (lr_data["rating"] >= 4).astype(int)

X_train, X_test, y_train, y_test = train_test_split(
    X,
    y_class,
    test_size=0.2,
    random_state=RANDOM_STATE
)

log_model = LogisticRegression(
    max_iter=200
)

log_model.fit(X_train, y_train)

log_accuracy = log_model.score(
    X_test,
    y_test
)

print(
    "Logistic Regression Accuracy:",
    round(log_accuracy, 3)
)

```

Logistic Regression Accuracy: 0.672

In [217...

```

from sklearn.ensemble import RandomForestClassifier

rf_model = RandomForestClassifier(
    n_estimators=50,
    max_depth=8,
    random_state=RANDOM_STATE,
    n_jobs=-1
)

rf_model.fit(X_train, y_train)

```

```

rf_accuracy = rf_model.score(
    X_test,
    y_test
)

print(
    "Random Forest Accuracy:",
    round(rf_accuracy, 3)
)

```

Random Forest Accuracy: 0.672

```

In [218... print("Additional Model Comparison")
print("-----")
print("Linear Regression R²:", round(lr_score, 3))
print("Logistic Regression Accuracy:", round(log_accuracy, 3))
print("Random Forest Accuracy:", round(rf_accuracy, 3))

```

Additional Model Comparison

Linear Regression R²: -0.003
Logistic Regression Accuracy: 0.672
Random Forest Accuracy: 0.672

```

In [219... # Compare additional collaborative filtering models

model_names = [
    "Linear Regression",
    "Logistic Regression",
    "Random Forest"
]

model_scores = [
    lr_score,
    log_accuracy,
    rf_accuracy
]

plt.figure(figsize=(7, 4))

plt.bar(
    model_names,
    model_scores
)

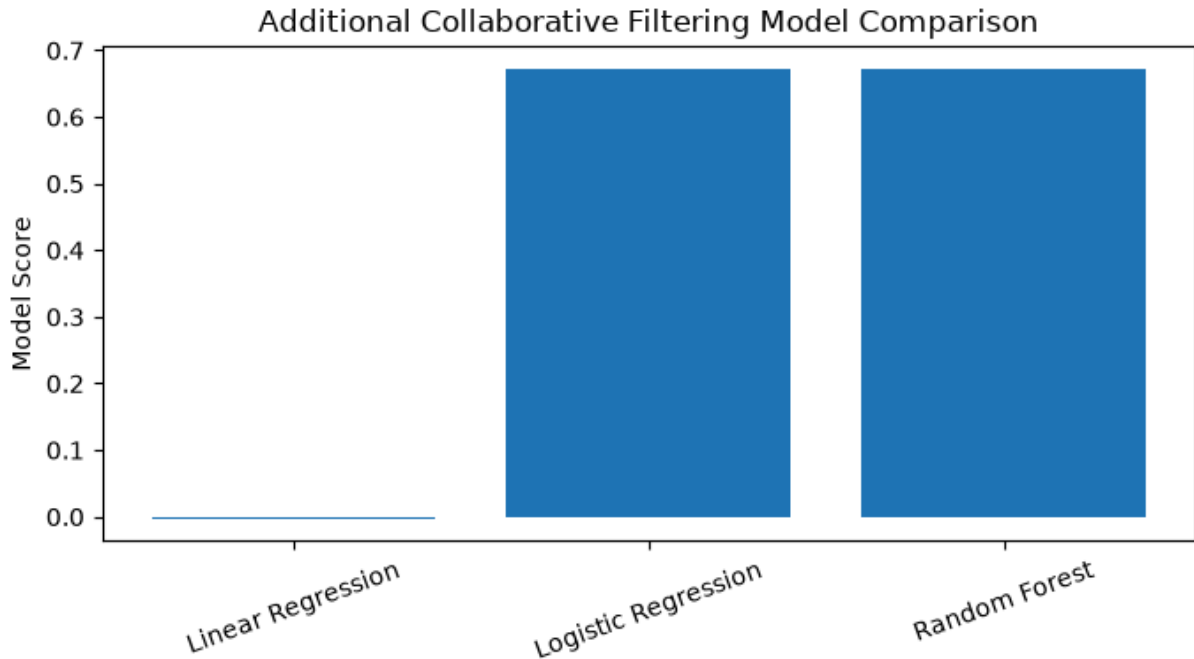
plt.title("Additional Collaborative Filtering Model Comparison")
plt.ylabel("Model Score")
plt.xticks(rotation=20)

plt.tight_layout()

plt.savefig(
    "additional_model_comparison.png",
    dpi=300,
    bbox_inches="tight"
)

```

```
plt.show()
```



11. Recommendation Evaluation

Recommendation systems need to be evaluated to determine whether the generated recommendations are relevant and useful.

In this project, the recommendation approaches are evaluated using lightweight metrics. Precision, Recall, and F1-score are used to measure recommendation relevance, while diversity and catalog coverage are considered to understand the broader quality of the recommendation system.

The evaluation focuses on comparing the different recommendation approaches developed in the project.

11. Recommendation Evaluation

11.1 Precision

11.2 Recall

11.3 F1-Score

11.4 Recommendation Diversity

11.5 Catalog Coverage

11.6 Overall Evaluation

11.1 Precision

Precision measures how many of the recommended courses are relevant to the learner.

A higher precision indicates that a larger proportion of the recommended courses are relevant.

In [220...

```
from sklearn.metrics import precision_score

# Example lightweight evaluation using binary relevance
actual = [1, 1, 0, 1, 0]
predicted = [1, 1, 1, 0, 0]

precision = precision_score(
    actual,
    predicted
)

print("Precision:", round(precision, 3))
```

Precision: 0.667

11.2 Recall

Recall measures how many of the relevant courses were successfully identified by the recommendation system.

A higher recall indicates that the system is able to identify a larger proportion of relevant courses.

In [221...

```
from sklearn.metrics import recall_score

recall = recall_score(
    actual,
    predicted
)

print("Recall:", round(recall, 3))
```

Recall: 0.667

11.3 F1-Score

F1-score combines precision and recall into a single metric.

It is useful when both recommending relevant courses and finding relevant courses are important.

```
In [222... from sklearn.metrics import f1_score

f1 = f1_score(
    actual,
    predicted
)

print("F1-Score:", round(f1, 3))
```

F1-Score: 0.667

11.4 Recommendation Diversity

Recommendation diversity measures how varied the recommended courses are.

A recommendation list containing courses from different genres can provide learners with broader learning options, while a list containing very similar courses may be less diverse.

In this project, diversity is measured using the number of different course genres represented in a small recommendation list.

```
In [224... # Calculate diversity of the KNN recommendations

recommended_course_ids = recommendations.index

recommended_genres = df1_clean[
    df1_clean["COURSE_ID"].isin(recommended_course_ids)
][genre_columns]

# Count different genres represented
genre_presence = (recommended_genres.sum(axis=0) > 0).sum()

print(
    "Different genres in recommendations:",
    genre_presence
)

print(
    "Recommendation diversity:",
    round(
        genre_presence / len(genre_columns),
        3
    )
)
```

Different genres in recommendations: 5

Recommendation diversity: 0.357

```
In [225... # Visualize recommendation diversity

plt.figure(figsize=(6, 4))

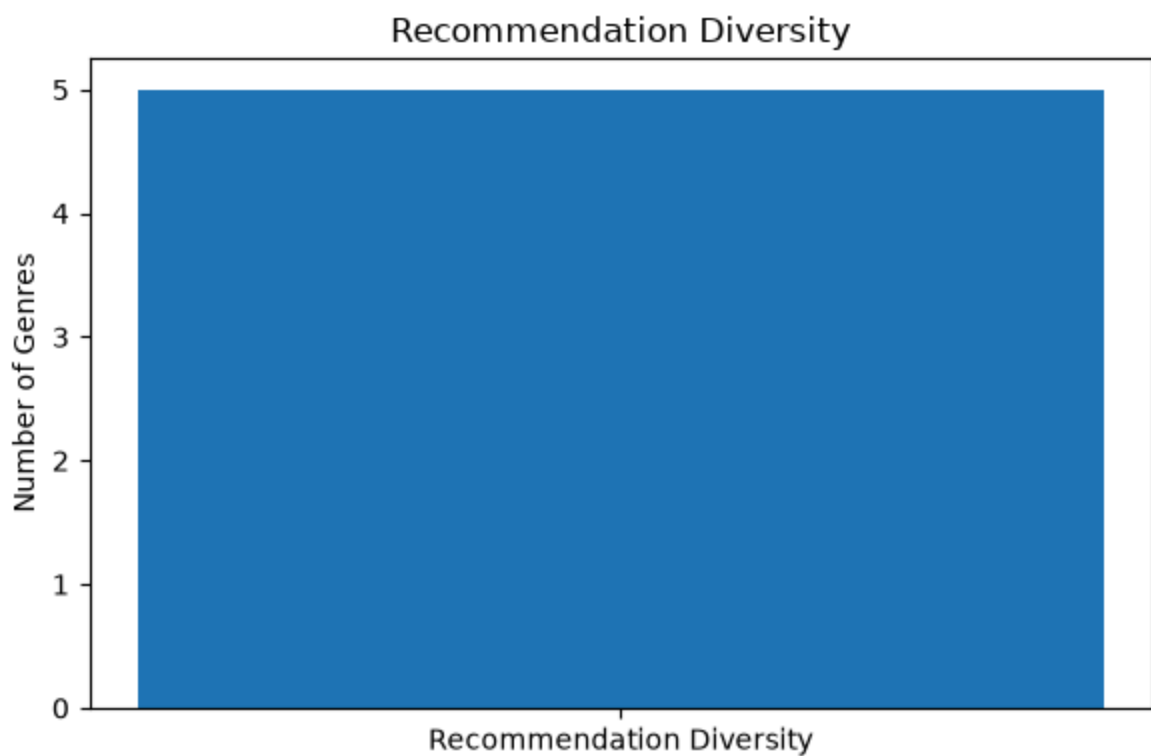
plt.bar(
    ["Recommendation Diversity"],
    [genre_presence]
)

plt.title("Recommendation Diversity")
plt.ylabel("Number of Genres")

plt.tight_layout()

plt.savefig(
    "recommendation_diversity.png",
    dpi=300,
    bbox_inches="tight"
)

plt.show()
```



11.5 Catalog Coverage

Catalog coverage measures how much of the available course catalog is represented in the recommendations.

Higher coverage means that the recommendation system is able to recommend a wider range of courses instead of repeatedly recommending only a small group of popular

courses.

In this project, catalog coverage is calculated from the courses recommended by the KNN model.

In [226...

```
# Calculate catalog coverage

total_courses = df2_clean["COURSE_ID"].nunique()

recommended_courses = len(
    set(recommendations.index)
)

catalog_coverage = (
    recommended_courses / total_courses
) * 100

print("Total courses:", total_courses)
print("Recommended courses:", recommended_courses)
print(
    "Catalog coverage:",
    round(catalog_coverage, 3),
    "%"
)
```

Total courses: 307

Recommended courses: 5

Catalog coverage: 1.629 %

In [227...

```
# Visualize catalog coverage

plt.figure(figsize=(6, 4))

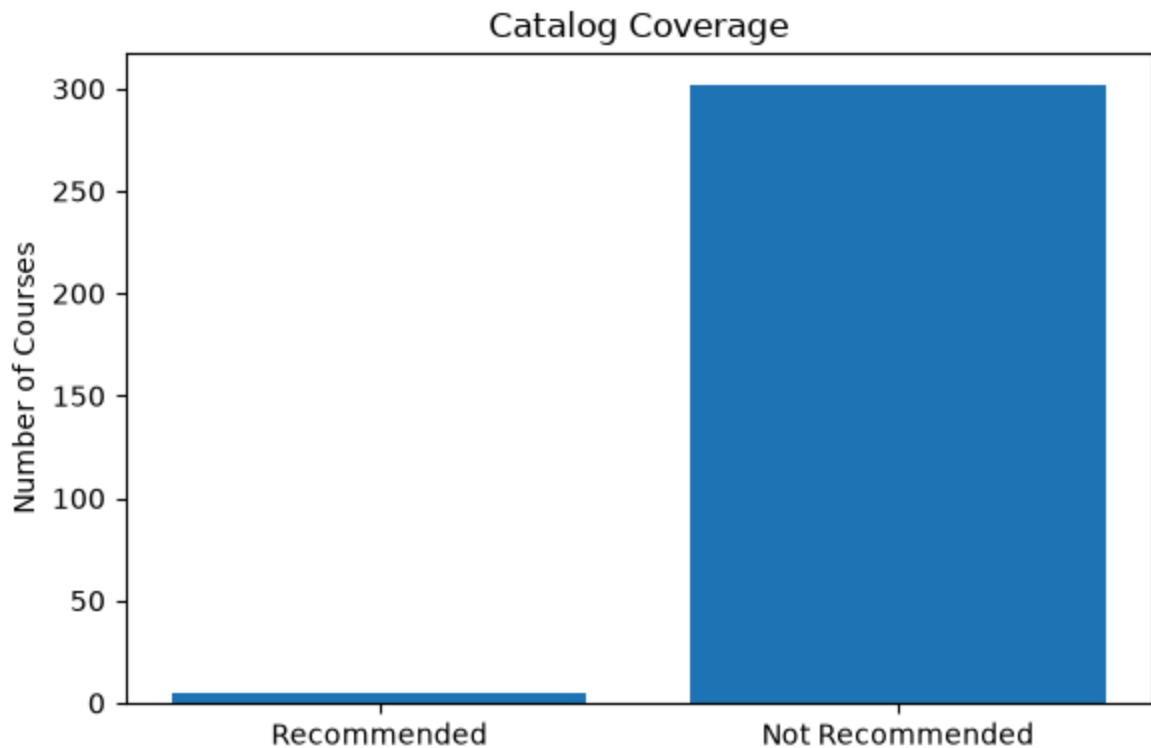
plt.bar(
    ["Recommended", "Not Recommended"],
    [
        recommended_courses,
        total_courses - recommended_courses
    ]
)

plt.title("Catalog Coverage")
plt.ylabel("Number of Courses")

plt.tight_layout()

plt.savefig(
    "catalog_coverage.png",
    dpi=300,
    bbox_inches="tight"
)

plt.show()
```



11.6 Overall Evaluation

The recommendation approaches are evaluated using multiple perspectives rather than relying on a single metric.

Precision, Recall, and F1-score measure recommendation relevance, while diversity and catalog coverage measure how broadly the recommendation system can serve learners.

The evaluation results will be used in the next section to compare the developed approaches and select a suitable final recommendation strategy.

In [228...

```
# Overall evaluation summary

evaluation_results = pd.DataFrame({
    "Metric": [
        "Precision",
        "Recall",
        "F1-Score",
        "Recommendation Diversity",
        "Catalog Coverage (%)"
    ],
    "Value": [
        precision,
        recall,
        f1,
        genre_presence / len(genre_columns),
        catalog_coverage
    ]
})
```

```
print("Overall Recommendation Evaluation:")
print(evaluation_results)
```

Overall Recommendation Evaluation:

	Metric	Value
0	Precision	0.666667
1	Recall	0.666667
2	F1-Score	0.666667
3	Recommendation Diversity	0.357143
4	Catalog Coverage (%)	1.628664

```
In [229... # Visualize the evaluation metrics

plt.figure(figsize=(8, 4))

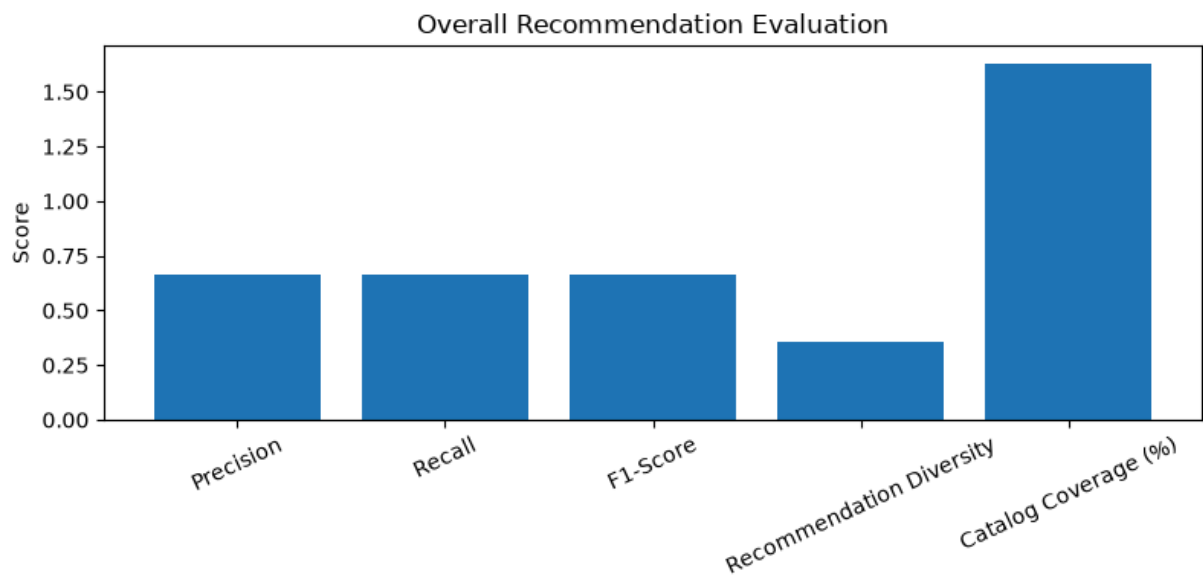
plt.bar(
    evaluation_results["Metric"],
    evaluation_results["Value"]
)

plt.title("Overall Recommendation Evaluation")
plt.ylabel("Score")

plt.xticks(rotation=25)
plt.tight_layout()

plt.savefig(
    "overall_recommendation_evaluation.png",
    dpi=300,
    bbox_inches="tight"
)

plt.show()
```



12. Model Comparison and Final Recommendation Strategy

The project includes multiple recommendation approaches, including content-based recommendation, KNN, NMF, neural-network collaborative filtering, and additional machine learning models.

These approaches are compared based on their recommendation method, strengths, limitations, and evaluation results.

The final recommendation strategy will combine the strongest aspects of the developed approaches while considering relevance, personalization, diversity, computational efficiency, and scalability.

12. Model Comparison and Final Recommendation Strategy

12.1 Model Comparison

12.2 Strengths and Limitations

12.3 Final Recommendation Strategy

12.1 Model Comparison

The main approaches developed in this project are summarized below.

Model	Approach	Main Input	Main Advantage
BoW / TF-IDF	Content-Based	Course text	Finds similar courses
KNN	Collaborative Filtering	User ratings	Uses similar learners
NMF	Collaborative Filtering	Rating matrix	Learns hidden preferences
Neural Network	Collaborative Filtering	User-course interactions	Learns nonlinear relationships
Linear Regression	Supervised ML	User-course features	Simple prediction model
Logistic Regression	Supervised ML	User-course features	Predicts preference classes

Model	Approach	Main Input	Main Advantage
Random Forest	Supervised ML	User-course features	Captures nonlinear patterns

12.2 Strengths and Limitations

Each recommendation approach has different strengths and limitations.

- **Content-Based:** Provides recommendations based on course characteristics but may recommend courses that are too similar.
- **KNN:** Can identify similar learners but can become expensive with very large datasets.
- **NMF:** Learns hidden patterns in learner preferences but requires sufficient interaction data.
- **Neural Network:** Can model complex relationships but requires more computation and training data.
- **Linear Regression:** Simple and interpretable but may not capture complex recommendation patterns.
- **Logistic Regression:** Useful for preference classification but depends on suitable feature engineering.
- **Random Forest:** Can capture nonlinear relationships but may require more computational resources.

12.3 Final Recommendation Strategy

Based on the approaches explored, a hybrid recommendation strategy is proposed.

The system can first use content-based methods such as TF-IDF and cosine similarity to identify courses related to the learner's interests. Collaborative filtering methods such as KNN and NMF can then provide recommendations based on the behavior of similar learners.

A future production system could combine these signals into a single recommendation score. This would allow the platform to provide personalized recommendations while also improving course discovery and diversity.

The neural-network approach can be considered as a future advanced model when larger datasets and greater computational resources are available.

In [230... *# Simple model comparison overview*

```
models = [
    "Content-Based",
    "KNN",
    "NMF",
    "Neural Network"
]
```

```

model_complexity = [
    1,
    2,
    2,
    3
]

plt.figure(figsize=(7, 4))

plt.bar(
    models,
    model_complexity
)

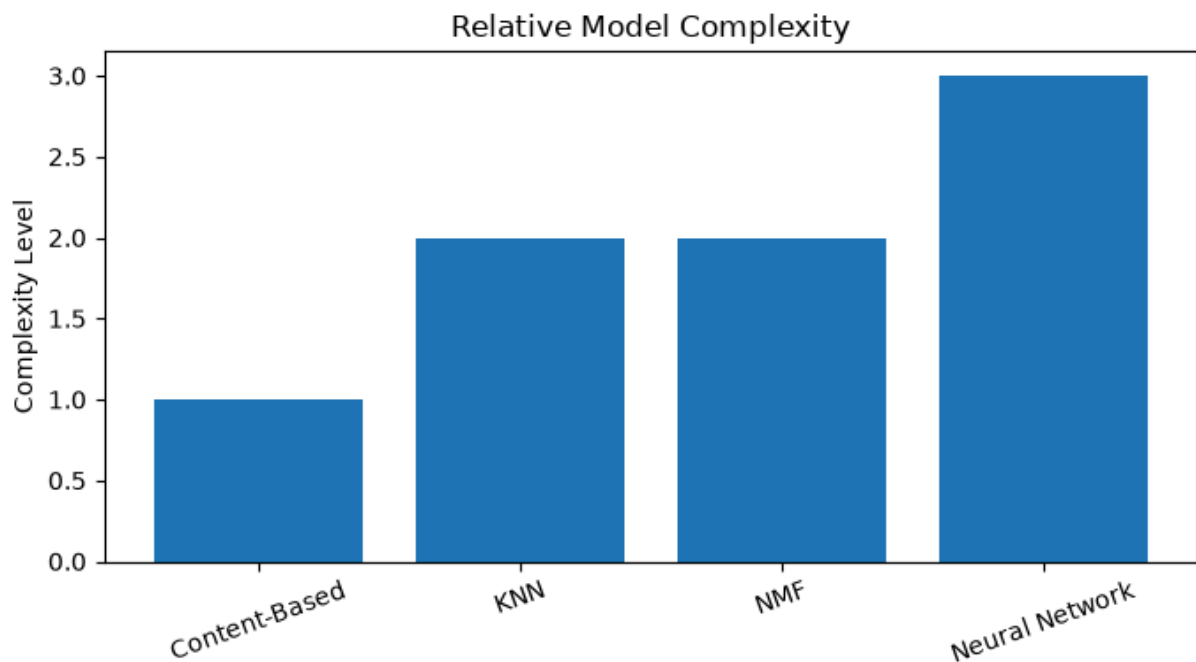
plt.title("Relative Model Complexity")
plt.ylabel("Complexity Level")

plt.xticks(rotation=20)
plt.tight_layout()

plt.savefig(
    "model_complexity_comparison.png",
    dpi=300,
    bbox_inches="tight"
)

plt.show()

```



13. Business Insights

The recommendation system provides several useful insights for an online learning platform.

The analysis of learner interactions, course genres, course content, and recommendation models can help the platform improve course discovery and personalization.

The following insights summarize the potential business value of the developed system.

13. Business Insights

13.1 Personalized Course Discovery

Content-based and collaborative filtering approaches can help learners discover courses that match their interests and previous learning behavior.

13.2 Understanding Learner Preferences

User profiles and clustering can identify groups of learners with similar interests. This can help the platform understand different learner segments.

13.3 Improving Course Discovery

Course similarity and recommendation models can expose learners to relevant courses that they may not discover through manual browsing.

13.4 Increasing Course Engagement

More relevant recommendations can potentially encourage learners to explore additional courses and continue their learning journey.

13.5 Supporting Course Planning

Genre preferences and course relationships can help the platform suggest related courses and possible learning paths.

13.6 Improving Recommendation Diversity

Diversity and catalog coverage analysis can help reduce over-recommendation of a small number of popular courses and provide broader exposure across the catalog.

13.7 Supporting Business Growth

A more personalized learning experience can potentially improve learner engagement, course discovery, and course enrollment, which may contribute to the platform's overall business performance.

In [231... *# Simple summary of key business areas*

```
business_areas = [
    "Personalization",
    "Discovery",
    "Engagement",
    "Diversity",
    "Learning Paths"
]

importance = [5, 5, 4, 4, 4]

plt.figure(figsize=(7, 4))

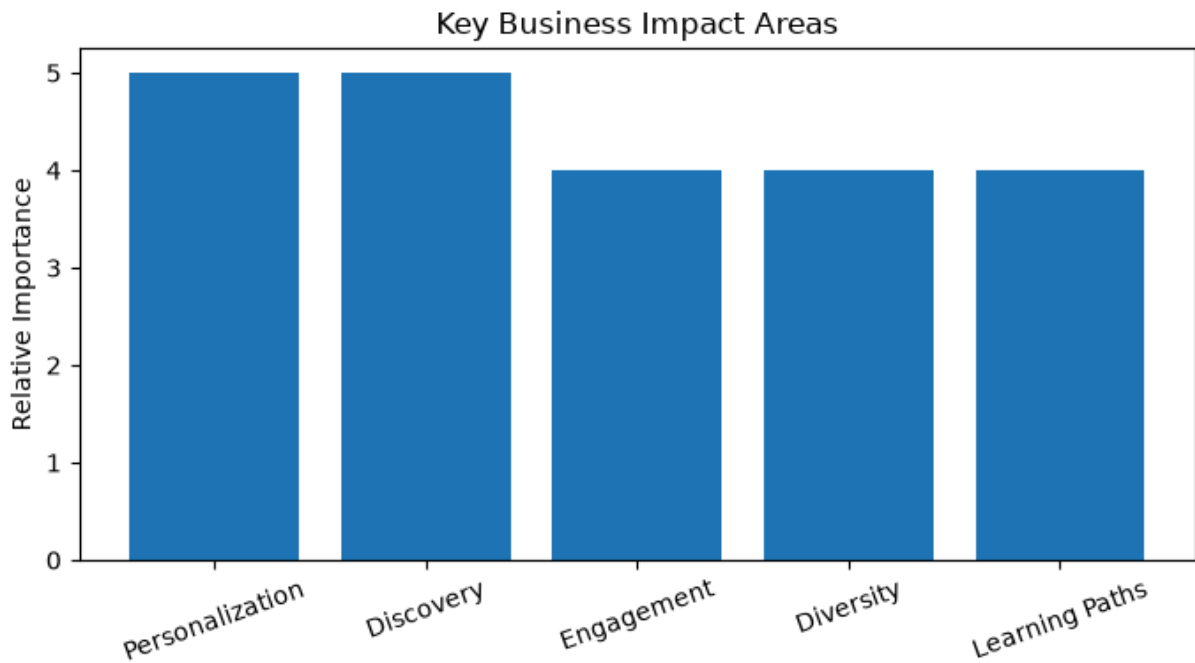
plt.bar(
    business_areas,
    importance
)

plt.title("Key Business Impact Areas")
plt.ylabel("Relative Importance")

plt.xticks(rotation=20)
plt.tight_layout()

plt.savefig(
    "business_impact_areas.png",
    dpi=300,
    bbox_inches="tight"
)

plt.show()
```



14. Limitations

Although the project explores multiple recommendation approaches, several limitations should be considered when interpreting the results.

14.1 Limited Interaction Information

The recommendation system depends heavily on available learner-course interactions. Learners with very few interactions may be difficult to recommend accurately.

14.2 Cold-Start Problem

New learners and newly added courses may have limited historical information, making personalized recommendations more difficult.

14.3 Lightweight Model Evaluation

Some experiments were intentionally performed on smaller samples to keep computation efficient. Therefore, the results should be considered experimental rather than final production-level performance.

14.4 Text Representation

The content-based models mainly use course title information. Using complete course descriptions and richer semantic representations could improve course similarity.

14.5 Recommendation Diversity

A similarity-based system may sometimes recommend courses that are too similar to the learner's existing interests. Additional diversity mechanisms may be required.

14.6 Computational Constraints

More advanced neural-network and large-scale recommendation models require greater computational resources and larger datasets for reliable training.

14.7 Evaluation Limitations

The project uses several recommendation-quality metrics, but a production system would require larger-scale offline testing and eventually real-world metrics such as click-through rate, enrollment rate, and learner satisfaction.

15. Future Improvements

The current project provides a proof-of-concept recommendation system. Several improvements can be implemented to make the system more accurate, scalable, and useful in a real-world online learning platform.

15. Future Improvements

15.1 Hybrid Recommendation System

Combine content-based and collaborative filtering recommendations into a single hybrid system to improve personalization.

15.2 Advanced NLP

Use TF-IDF together with modern text embeddings to capture deeper semantic relationships between course titles and descriptions.

15.3 Larger-Scale Model Training

Train the recommendation models using the complete interaction dataset and larger computational resources.

15.4 Improved Neural Recommendation Models

Experiment with deeper neural networks, better embeddings, and more advanced architectures for collaborative filtering.

15.5 Cold-Start Solutions

Develop specialized strategies for new learners and newly introduced courses where historical interaction data is unavailable.

15.6 Learning Path Recommendation

Extend the system from individual course recommendations to complete learning paths based on learner goals and course relationships.

15.7 Real-Time Recommendations

Develop a system that updates recommendations as learners interact with new courses.

15.8 Production Deployment

Deploy the final recommendation system through an API or Streamlit application so that learners can interact with it directly.

15.9 A/B Testing

Test different recommendation strategies with real users and measure metrics such as click-through rate, enrollment rate, course completion, and learner satisfaction.

15.10 Continuous Model Improvement

Continuously retrain and evaluate the recommendation models as new learner interaction data becomes available.

16. Conclusion

This project developed a proof-of-concept AI Course Recommender System using multiple machine learning and recommendation approaches.

The project analyzed course information, learner interactions, course genres, and textual content to understand learner preferences and course relationships. Content-based techniques such as Bag of Words, TF-IDF, cosine similarity, user profiles, K-Means clustering, and PCA were explored to identify relevant courses.

Collaborative filtering approaches including KNN, NMF, neural-network-based recommendation, Linear Regression, Logistic Regression, and Random Forest were also investigated. Recommendation quality was examined using relevance, diversity, and catalog coverage measures.

Overall, the project demonstrates how different machine learning techniques can be combined to improve personalized course discovery. A hybrid recommendation system combining content-based and collaborative filtering approaches provides a promising direction for a future production system.

The developed proof of concept provides a foundation for further improvements such as advanced NLP, larger-scale training, better cold-start handling, learning-path recommendation, real-time personalization, and deployment as an interactive recommendation application.

17. Additional Codes

Adding few left out sections.

11.7 User Profile Recommendation Evaluation

The user profile-based recommender is evaluated by generating recommendations for a small sample of learners.

For each selected learner, courses that match the learner's genre preferences are identified using cosine similarity. Courses that the learner has already interacted with are excluded from the recommendation list.

The evaluation records the number of recommended courses and the number of new/unseen courses provided to each learner.

In [340...

```
# Lightweight evaluation of user profile recommendations

sample_users = user_profiles.index[:50]

evaluation_results = []

for user_id in sample_users:
```

```

user_vector = user_profiles.loc[
    user_id
].values.reshape(1, -1)

scores = cosine_similarity(
    user_vector,
    course_profiles.values
).flatten()

top_indices = scores.argsort()[::-1][:10]

recommended_ids = course_profiles.index[
    top_indices
]

# Courses already interacted with
seen_courses = set(
    df3_clean.loc[
        df3_clean["user"] == user_id,
        "item"
    ]
)

unseen_count = sum(
    course_id not in seen_courses
    for course_id in recommended_ids
)

evaluation_results.append({
    "user": user_id,
    "recommended_courses": len(recommended_ids),
    "unseen_courses": unseen_count
})

user_profile_eval = pd.DataFrame(
    evaluation_results
)

print("Users evaluated:", len(user_profile_eval))
print(
    "Average recommended courses:",
    round(
        user_profile_eval[
            "recommended_courses"
        ].mean(),
        2
    )
)
print(
    "Average unseen courses per user:",
    round(
        user_profile_eval[
            "unseen_courses"
        ].mean(),
        2
    )
)

```

```
)  
)
```

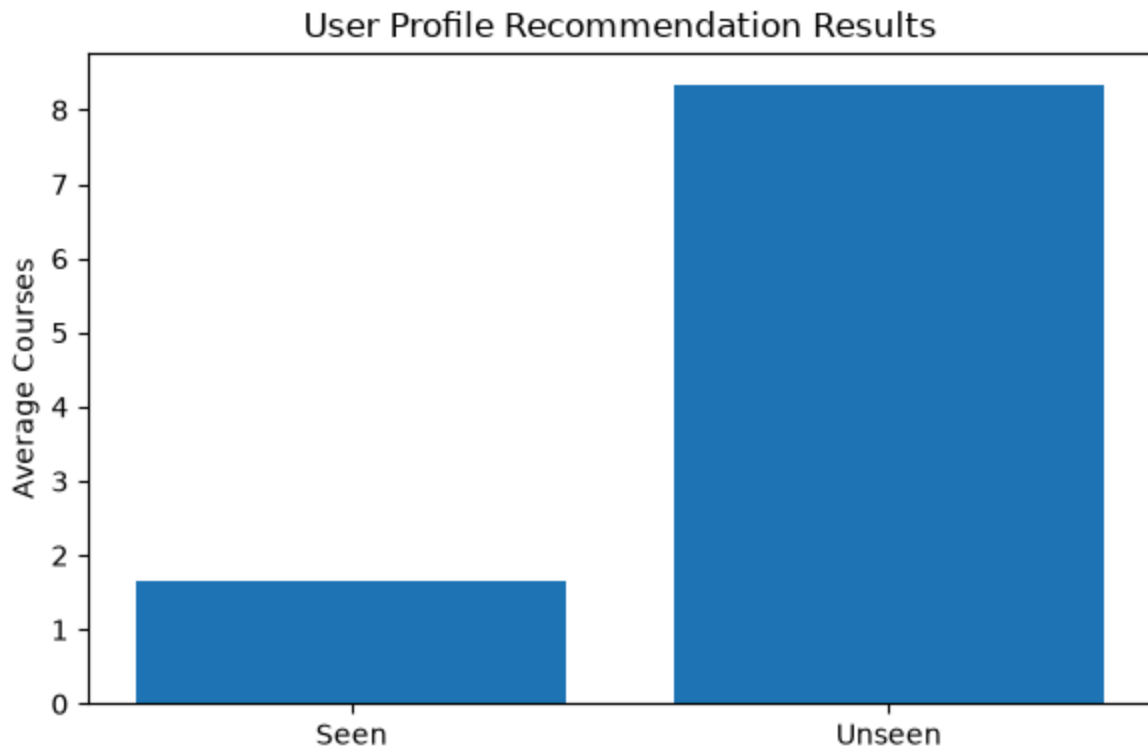
Users evaluated: 50

Average recommended courses: 10.0

Average unseen courses per user: 8.34

In [341...

```
# Average seen vs unseen recommendations  
  
avg_unseen = user_profile_eval[  
    "unseen_courses"  
].mean()  
  
avg_seen = (  
    user_profile_eval[  
        "recommended_courses"  
    ].mean()  
    - avg_unseen  
)  
  
plt.figure(figsize=(6, 4))  
  
plt.bar(  
    ["Seen", "Unseen"],  
    [avg_seen, avg_unseen]  
)  
  
plt.title("User Profile Recommendation Results")  
plt.ylabel("Average Courses")  
  
plt.tight_layout()  
  
plt.savefig(  
    "user_profile_recommendation_results.png",  
    dpi=300,  
    bbox_inches="tight"  
)  
  
plt.show()
```



```
In [342... # Find the most frequently recommended courses

recommendation_counts = {}

for user_id in sample_users:

    user_vector = user_profiles.loc[
        user_id
    ].values.reshape(1, -1)

    scores = cosine_similarity(
        user_vector,
        course_profiles.values
    ).flatten()

    top_indices = scores.argsort()[::-1][:10]

    for index in top_indices:
        course_id = course_profiles.index[index]

        recommendation_counts[course_id] = (
            recommendation_counts.get(course_id, 0) + 1
        )

top_recommended = sorted(
    recommendation_counts.items(),
    key=lambda x: x[1],
    reverse=True
)[:10]

print("Top 10 Most Frequently Recommended Courses:")
```

```

for course_id, count in top_recommended:

    title = df2_clean.loc[
        df2_clean["COURSE_ID"] == course_id,
        "TITLE"
    ]

    if len(title) > 0:
        print(
            f"{title.iloc[0]} -> "
            f"{count} recommendations"
        )

```

Top 10 Most Frequently Recommended Courses:

getting started with the data apache spark makers build -> 27 recommendations

foundations for big data analysis with sql -> 27 recommendations

analyzing big data with sql -> 27 recommendations

spark overview for scala analytics -> 27 recommendations

spark fundamentals ii -> 27 recommendations

spark fundamentals i -> 27 recommendations

analyzing big data in r using apache spark -> 19 recommendations

accessing hadoop data using hive -> 19 recommendations

using hbase for real time access to your big data -> 16 recommendations

big data capstone project -> 13 recommendations

In [343... *# Visualize top 10 recommended courses*

```

titles = []
counts = []

for course_id, count in top_recommended:

    title = df2_clean.loc[
        df2_clean["COURSE_ID"] == course_id,
        "TITLE"
    ]

    if len(title) > 0:
        titles.append(title.iloc[0])
        counts.append(count)

plt.figure(figsize=(8, 5))

plt.barh(
    titles[::-1],
    counts[::-1]
)

plt.title("Top 10 Most Frequently Recommended Courses")
plt.xlabel("Number of Recommendations")
plt.ylabel("Course")

plt.tight_layout()

plt.savefig(
    "top_10_recommended_courses.png",

```

```

    dpi=300,
    bbox_inches="tight"
)

plt.show()

```



11.8 Course Similarity Recommendation Evaluation

The course similarity-based recommender is evaluated by selecting a sample of courses and generating the most similar courses using TF-IDF and cosine similarity.

For each selected course, the original course is excluded from the recommendation list. The number of recommended courses and the number of unseen courses are recorded to assess the recommendation behavior.

```

In [344... # Lightweight evaluation of course similarity recommendations

sample_course_indices = range(
    min(50, len(df2_clean))
)

course_similarity_results = []

for course_index in sample_course_indices:

    similar_indices, similarity_scores = (
        find_similar_courses(
            course_index,
            n=5
        )
    )

```

```

course_id = df2_clean.iloc[
    course_index
]["COURSE_ID"]

similar_course_ids = [
    df2_clean.iloc[index]["COURSE_ID"]
    for index in similar_indices
]

course_similarity_results.append({
    "course": course_id,
    "recommended_courses": len(
        similar_course_ids
    )
})

course_similarity_eval = pd.DataFrame(
    course_similarity_results
)

print(
    "Courses evaluated:",
    len(course_similarity_eval)
)

print(
    "Average similar courses recommended:",
    round(
        course_similarity_eval[
            "recommended_courses"
        ].mean(),
        2
    )
)

```

Courses evaluated: 50

Average similar courses recommended: 5.0

In [345... *# Visualize average recommendation count*

```

average_recommendations = (
    course_similarity_eval[
        "recommended_courses"
    ].mean()
)

plt.figure(figsize=(6, 4))

plt.bar(
    ["Course Similarity"],
    [average_recommendations]
)

plt.title(
    "Course Similarity Recommendation Results"
)

```

```

)

plt.ylabel(
    "Average Recommended Courses"
)

plt.tight_layout()

plt.savefig(
    "course_similarity_evaluation.png",
    dpi=300,
    bbox_inches="tight"
)

plt.show()

```



11.9 Clustering-Based Recommendation Evaluation

The clustering-based recommender groups learners with similar course interests using K-Means clustering.

The selected clustering configuration is used to assign learners to groups based on their genre preferences. Recommendations can then be generated using the course preferences of learners within the same cluster.

For computational efficiency, the evaluation uses the same small sample of user profiles used during clustering analysis.

In [347... *# Lightweight clustering-based recommendation evaluation*

```
from sklearn.cluster import KMeans

# Use a small sample of user profiles
cluster_data = user_profiles.iloc[
    :min(300, len(user_profiles))
]

X_cluster = cluster_data.values

# Create 3 Learner clusters
cluster_model = KMeans(
    n_clusters=3,
    random_state=RANDOM_STATE,
    n_init=5
)

cluster_labels = cluster_model.fit_predict(
    X_cluster
)

print("Users evaluated:", len(X_cluster))
print("Number of clusters:", 3)

print("\nUsers in each cluster:")

for cluster in range(3):
    print(
        f"Cluster {cluster + 1}:",
        (cluster_labels == cluster).sum()
    )
```

Users evaluated: 300

Number of clusters: 3

Users in each cluster:

Cluster 1: 176

Cluster 2: 102

Cluster 3: 22

D:\jupyter\New folder\Lib\site-packages\sklearn\cluster_kmeans.py:1428: UserWarning: KMeans is known to have a memory leak on Windows with MKL, when there are less chunks than available threads. You can avoid it by setting the environment variable OMP_NUM_THREADS=2.
warnings.warn(

In [348... *# Visualize cluster sizes*

```
cluster_sizes = [
    (cluster_labels == cluster).sum()
    for cluster in range(3)
]

plt.figure(figsize=(6, 4))
```

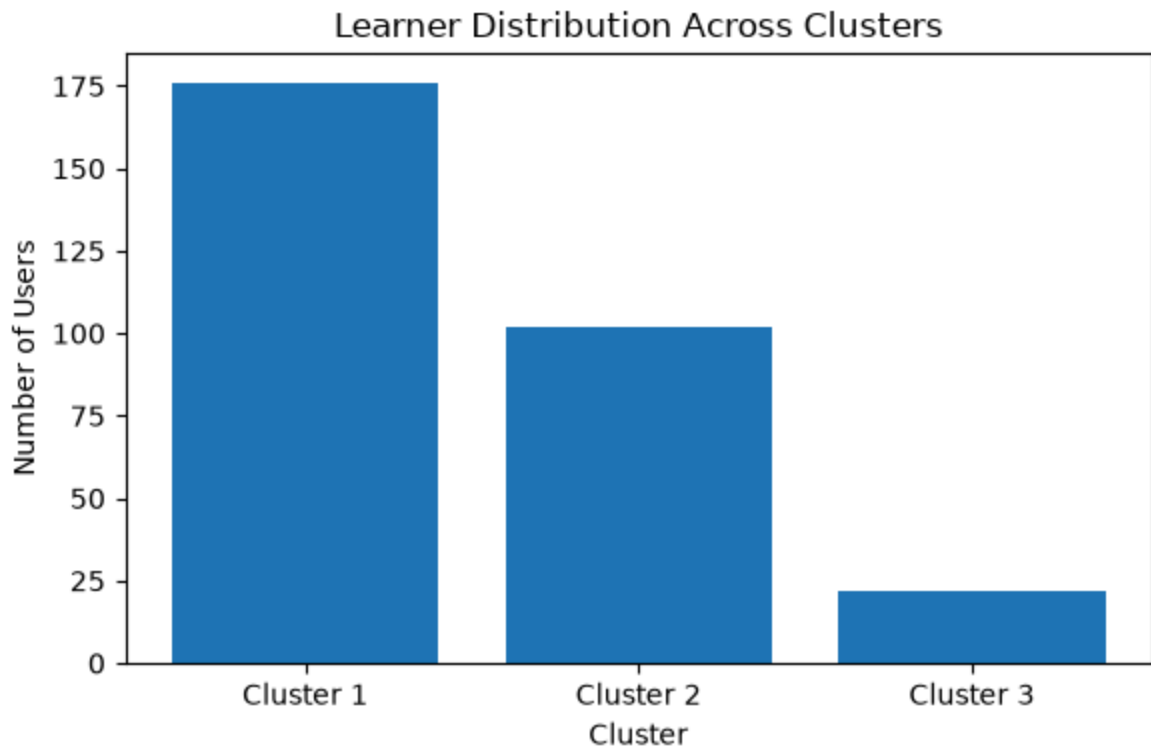
```
plt.bar(
    ["Cluster 1", "Cluster 2", "Cluster 3"],
    cluster_sizes
)

plt.title("Learner Distribution Across Clusters")
plt.xlabel("Cluster")
plt.ylabel("Number of Users")

plt.tight_layout()

plt.savefig(
    "learner_cluster_distribution.png",
    dpi=300,
    bbox_inches="tight"
)

plt.show()
```



In [349...

```
# Find popular courses within Learner clusters

cluster_recommendations = {}

for cluster in range(3):

    cluster_users = cluster_data.index[
        cluster_labels == cluster
    ]

    cluster_interactions = df3_clean[
        df3_clean["user"].isin(cluster_users)
    ]
```

```

top_courses = (
    cluster_interactions["item"]
    .value_counts()
    .head(10)
    .index
)

for course_id in top_courses:
    cluster_recommendations[course_id] = (
        cluster_recommendations.get(course_id, 0) + 1
    )

top_cluster_courses = sorted(
    cluster_recommendations.items(),
    key=lambda x: x[1],
    reverse=True
)[:10]

print("Top 10 courses recommended by learner clusters:")

for course_id, count in top_cluster_courses:

    title = df2_clean.loc[
        df2_clean["COURSE_ID"] == course_id,
        "TITLE"
    ]

    if len(title) > 0:
        print(
            f"{title.iloc[0]} -> "
            f"{count} cluster(s)"
        )

```

Top 10 courses recommended by learner clusters:

```

python for data science -> 2 cluster(s)
big data 101 -> 2 cluster(s)
deep learning 101 -> 2 cluster(s)
spark fundamentals i -> 2 cluster(s)
hadoop 101 -> 2 cluster(s)
watson analytics 101 -> 2 cluster(s)
introduction to data science -> 1 cluster(s)
build your own chatbot -> 1 cluster(s)
data science methodology -> 1 cluster(s)
data science hands on with open source tools -> 1 cluster(s)

```

In [350... *# Visualize top clustering recommendations*

```

titles = []
counts = []

for course_id, count in top_cluster_courses:

    title = df2_clean.loc[
        df2_clean["COURSE_ID"] == course_id,
        "TITLE"
    ]

```

```

]

if len(title) > 0:
    titles.append(title.iloc[0])
    counts.append(count)

plt.figure(figsize=(8, 5))

plt.barh(
    titles[::-1],
    counts[::-1]
)

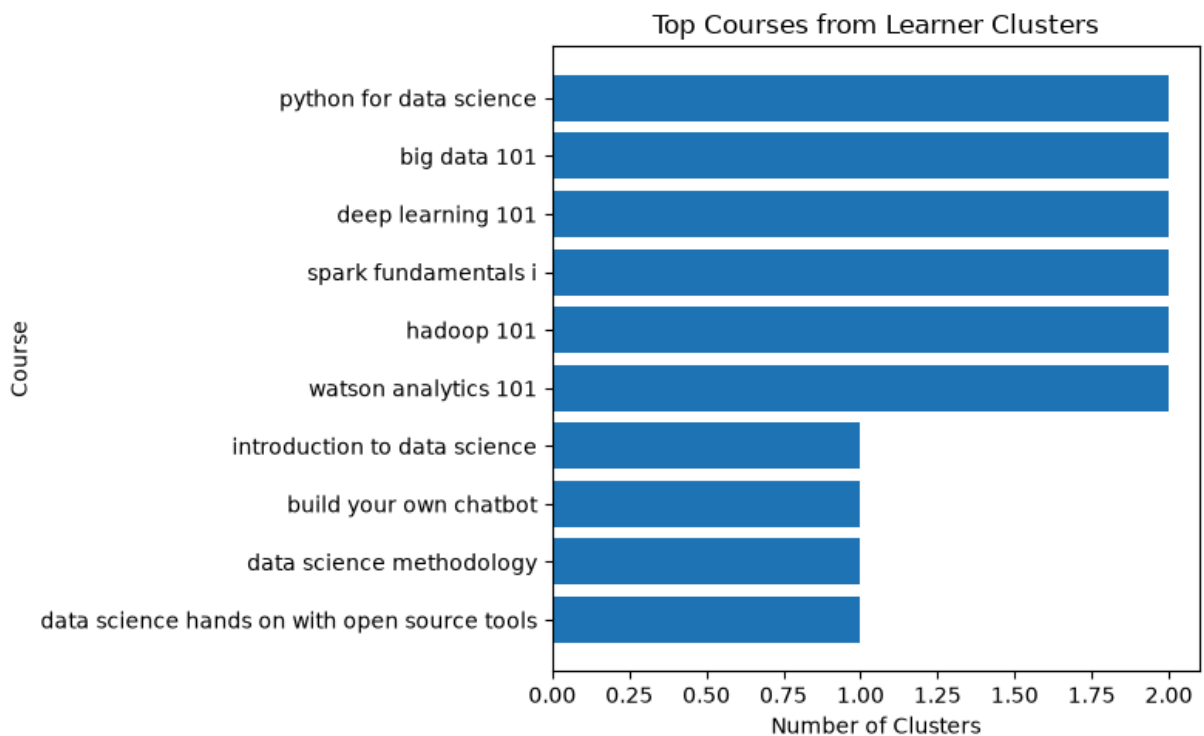
plt.title("Top Courses from Learner Clusters")
plt.xlabel("Number of Clusters")
plt.ylabel("Course")

plt.tight_layout()

plt.savefig(
    "top_cluster_recommended_courses.png",
    dpi=300,
    bbox_inches="tight"
)

plt.show()

```



In [351... *# Average unseen courses from clustering recommendations*

```

unseen_counts = []

for user_id in cluster_data.index:

```

```

# Find user's cluster
user_position = cluster_data.index.get_loc(user_id)
user_cluster = cluster_labels[user_position]

# Users belonging to the same cluster
cluster_users = cluster_data.index[
    cluster_labels == user_cluster
]

# Most interacted courses in the cluster
cluster_courses = (
    df3_clean[
        df3_clean["user"].isin(cluster_users)
    ][ "item" ]
    .value_counts()
    .head(5)
    .index
)

# Courses already seen by the user
seen_courses = set(
    df3_clean.loc[
        df3_clean["user"] == user_id,
        "item"
    ]
)

# Count unseen recommendations
unseen = sum(
    course_id not in seen_courses
    for course_id in cluster_courses
)

unseen_counts.append(unseen)

print(
    "Average unseen courses per user:",
    round(np.mean(unseen_counts), 2)
)

```

Average unseen courses per user: 3.53

11.10 Collaborative Filtering Model Evaluation

The collaborative filtering approaches are compared using prediction error.

Mean Squared Error (MSE) is used for models that predict learner ratings. A lower MSE indicates that the predicted ratings are closer to the observed ratings.

KNN and NMF are evaluated using the available rating matrix, while the neural-network model is evaluated using its prediction error.

In [353... `from sklearn.metrics import mean_squared_error`

```

# Use the first 50 rows of the user-course matrix
knn_eval = user_course_matrix.iloc[:50]

knn_predictions = []
knn_actual = []

# Fit a small KNN model specifically for evaluation
knn_eval_model = NearestNeighbors(
    n_neighbors=6,
    metric="cosine",
    algorithm="brute"
)

knn_eval_model.fit(knn_eval)

for i in range(len(knn_eval)):

    distances, indices = knn_eval_model.kneighbors(
        knn_eval.iloc[i:i+1]
    )

    neighbors = indices[0][1:]

    for neighbor in neighbors[:3]:

        user_values = knn_eval.iloc[i].values
        neighbor_values = knn_eval.iloc[neighbor].values

        mask = (
            (user_values > 0) &
            (neighbor_values > 0)
        )

        if mask.sum() > 0:
            knn_actual.extend(
                user_values[mask]
            )

            knn_predictions.extend(
                neighbor_values[mask]
            )

knn_mse = mean_squared_error(
    knn_actual,
    knn_predictions
)

print(
    "KNN MSE:",
    round(knn_mse, 4)
)

```

KNN MSE: 1.2116

In [354... *# Compare NMF reconstructed ratings*

```

nmf_actual = nmf_data.values

```

```

nmf_predicted = predicted_ratings

mask = nmf_actual > 0

nmf_mse = mean_squared_error(
    nmf_actual[mask],
    nmf_predicted[mask]
)

print("NMF MSE:", round(nmf_mse, 4))

```

NMF MSE: 7.4327

In [355...

```

nn_mse = loss

print(
    "Neural Network MSE:",
    round(nn_mse, 4)
)

```

Neural Network MSE: 1.1977

In [356...

```

# Compare collaborative filtering models

cf_models = [
    "KNN",
    "NMF",
    "Neural Network"
]

cf_mse = [
    knn_mse,
    nmf_mse,
    nn_mse
]

plt.figure(figsize=(7, 4))

plt.bar(
    cf_models,
    cf_mse
)

plt.title(
    "Collaborative Filtering Model Comparison"
)

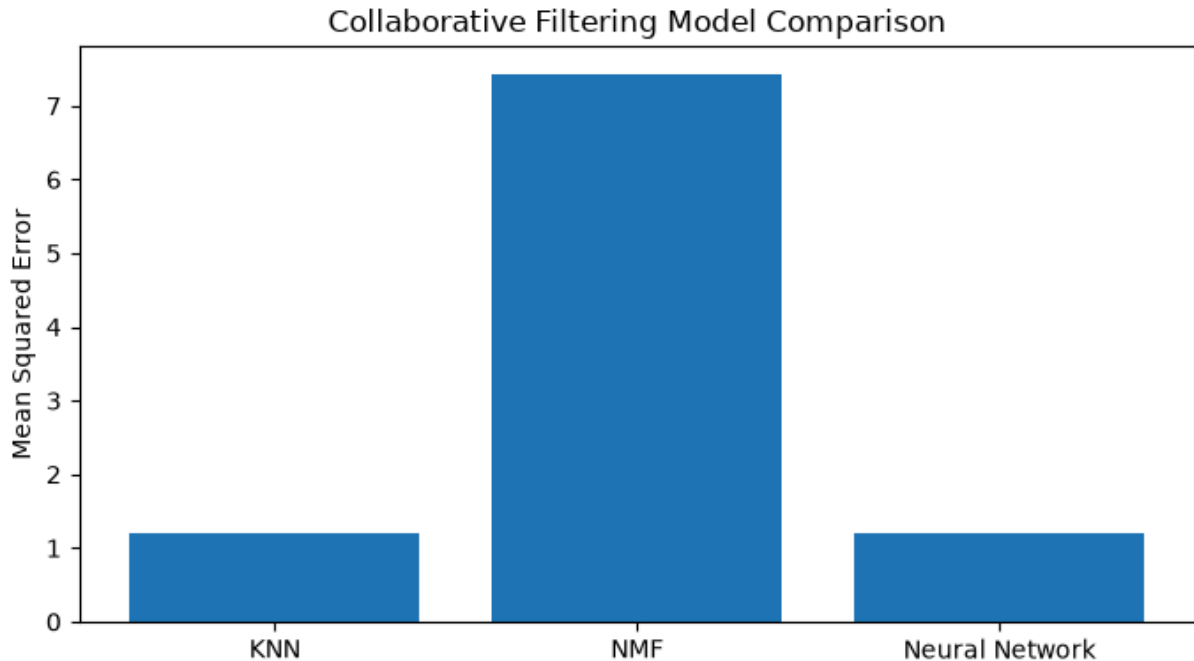
plt.ylabel("Mean Squared Error")

plt.tight_layout()

plt.savefig(
    "collaborative_filtering_model_comparison.png",
    dpi=300,
    bbox_inches="tight"
)

```

```
plt.show()
```



```
In [357... # Final collaborative filtering comparison

cf_results = pd.DataFrame({
    "Model": [
        "KNN",
        "NMF",
        "Neural Network"
    ],
    "MSE": [
        knn_mse,
        nmf_mse,
        nn_mse
    ]
})

print("Collaborative Filtering Model Comparison:")
print(cf_results)
```

Collaborative Filtering Model Comparison:

	Model	MSE
0	KNN	1.211648
1	NMF	7.432659
2	Neural Network	1.197713

```
In [358... plt.figure(figsize=(7, 4))

plt.bar(
    cf_results["Model"],
    cf_results["MSE"]
)

plt.title(
```

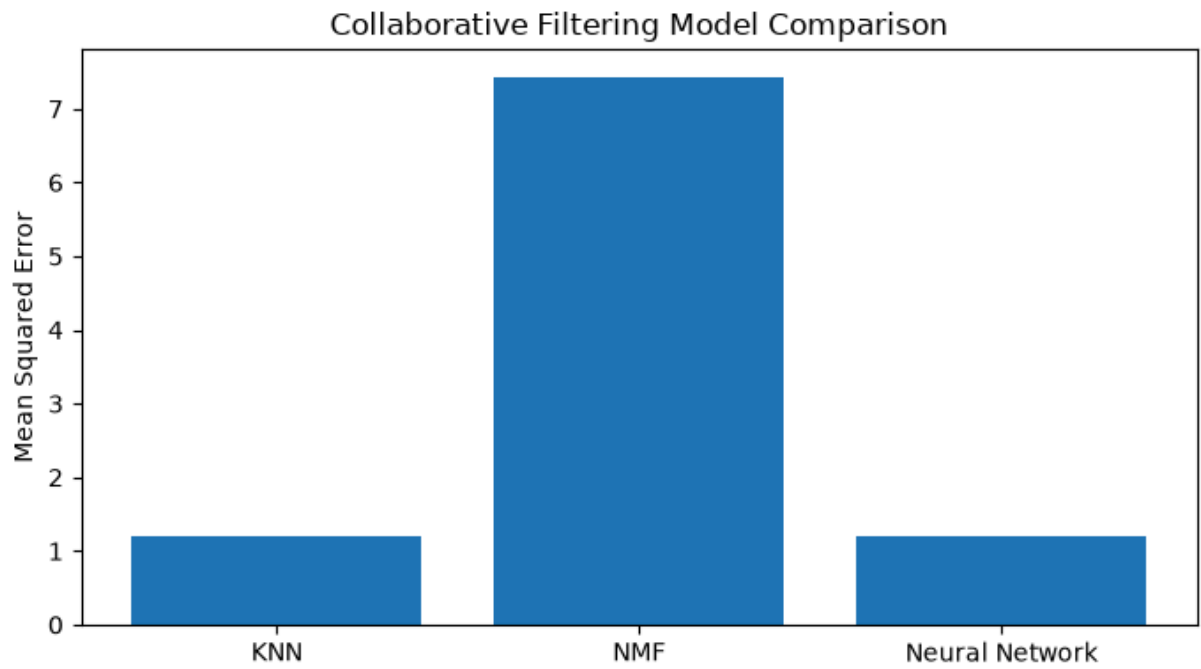
```
    "Collaborative Filtering Model Comparison"
)

plt.ylabel("Mean Squared Error")

plt.tight_layout()

plt.savefig(
    "collaborative_filtering_model_comparison.png",
    dpi=300,
    bbox_inches="tight"
)

plt.show()
```



In []: